

Service-oriented Software Engineering (SoSE)

Describing REST WS with Open API and Swagger

Prof. Marco Autili

Department of Information Engineering Computer Science and Mathematics

University of L'Aquila

marco.autili@univaq.it

<http://people.disim.univaq.it/marco.autili/>

What have we done? (from previous lectures)

When a service **has a complex interface** and is not a simple resource, it can be (more w.r.t. SOAP Services) **difficult to design a set of RESTful services to represent it**

There are **no standards for RESTful interface description** so service users must rely on informal documentation to understand the interface



- WADL (https://it.wikipedia.org/wiki/Web_Application_Description_Language)
- **OpenAPI** Specification (also known as **Swagger** Specification - https://it.wikipedia.org/wiki/Specifiche_OpenAPI)

What is Swagger?

It is for simplifying the design, development and documentation of APIs

- The power of the Swagger starts with the OpenAPI Specification, formerly known as the Swagger Specification, which is the industry standard for RESTful API design
- The specification permits to create a RESTful interface for easily developing and consuming an API by effectively mapping all the resources and operations associated with it

A curiosity

the OpenAPI Specification was donated to the Linux Foundation under the OpenAPI Initiative in 2015

We will see how to conveniently exploit Swagger to

1. document REST services
2. get a ready-to-use client web application that permit you to interact with REST services directly from a browser
3. Automatically generate a client application to junit test REST services

Ingredients

Java SE Development Kit 21

<https://www.oracle.com/java/technologies/javase/jdk21-archive-downloads.html>

IntelliJ IDEA Community Edition

<https://www.jetbrains.com/idea/download/?platform=windows&code=IIC§ion=windows>

Apache Tomcat® (v.11.0.7) [core - zip or tar.gz]

<https://tomcat.apache.org/download-11.cgi>

Apache CXF™ (v.4.1.1) [binary distribution - zip or tar.gz]

<http://cxf.apache.org/download.html>

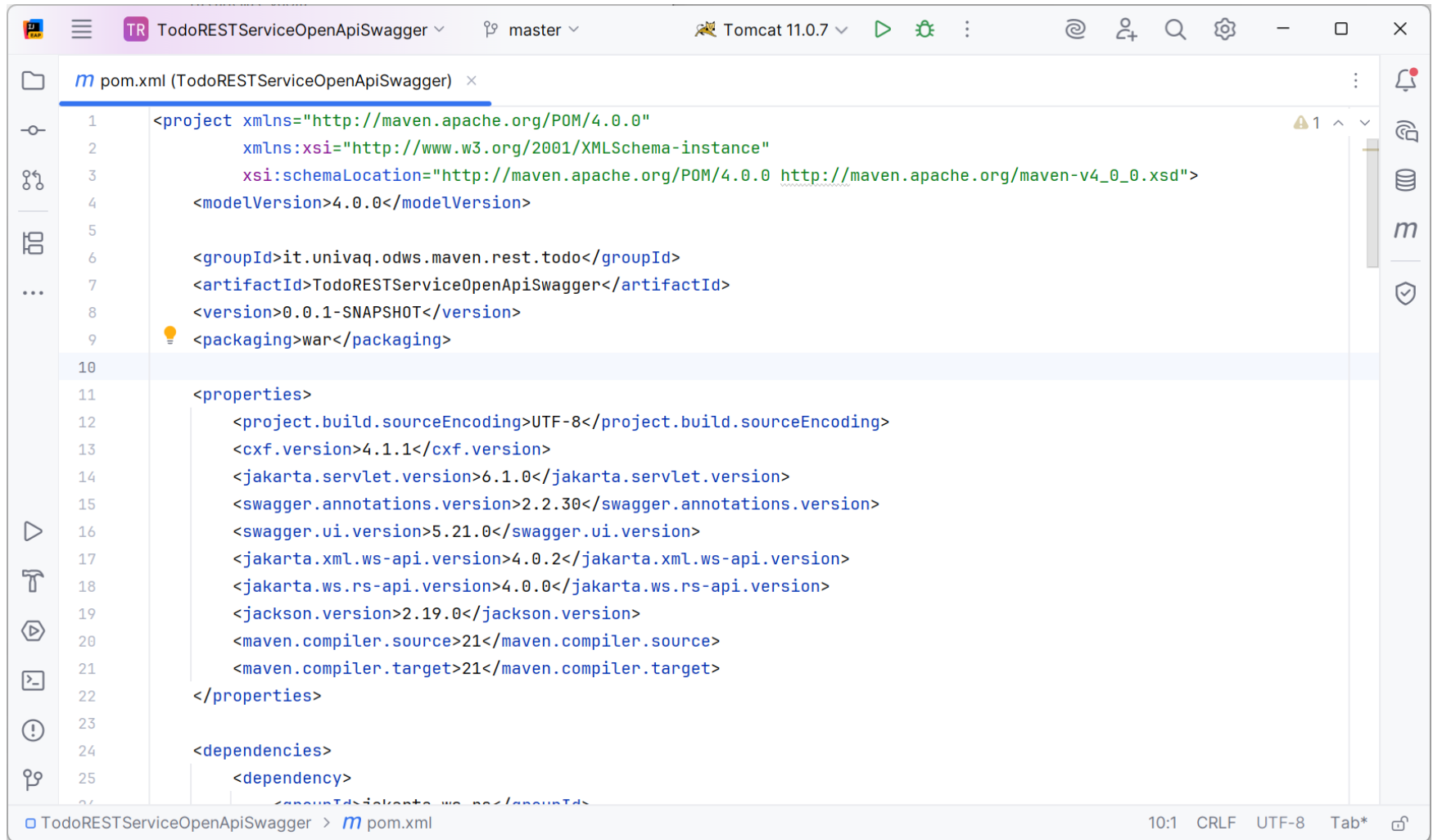
Apache Maven (v.3.9.9)

<https://maven.apache.org/download.cgi>

Swagger Editor

<https://editor.swagger.io/>

Create TodoRESTServiceOpenApiSwagger



The screenshot shows an IDE window with the title bar 'TR TodoRESTServiceOpenApiSwagger master Tomcat 11.0.7'. The editor displays the 'pom.xml' file for the project 'm pom.xml (TodoRESTServiceOpenApiSwagger)'. The XML content is as follows:

```
1 <project xmlns="http://maven.apache.org/POM/4.0.0"
2       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3       xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
4   <modelVersion>4.0.0</modelVersion>
5
6   <groupId>it.univaq.odws.maven.rest.todo</groupId>
7   <artifactId>TodoRESTServiceOpenApiSwagger</artifactId>
8   <version>0.0.1-SNAPSHOT</version>
9   <packaging>war</packaging>
10
11   <properties>
12     <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
13     <cxfr.version>4.1.1</cxfr.version>
14     <jakarta.servlet.version>6.1.0</jakarta.servlet.version>
15     <swagger.annotations.version>2.2.30</swagger.annotations.version>
16     <swagger.ui.version>5.21.0</swagger.ui.version>
17     <jakarta.xml.ws-api.version>4.0.2</jakarta.xml.ws-api.version>
18     <jakarta.ws.rs-api.version>4.0.0</jakarta.ws.rs-api.version>
19     <jackson.version>2.19.0</jackson.version>
20     <maven.compiler.source>21</maven.compiler.source>
21     <maven.compiler.target>21</maven.compiler.target>
22   </properties>
23
24   <dependencies>
25     <dependency>
```

The status bar at the bottom indicates '10:1 CRLF UTF-8 Tab*'.

Add OpenApi and Swagger dependencies

The screenshot shows an IDE window titled "pom.xml (TodoRESTServiceOpenApiSwagger)". The code is as follows:

```
1 <project xmlns="http://maven.apache.org/POM/4.0.0"
24 <dependencies>
30 </dependencies>
57 <!-- Open Api and Swagger -->
58 <!-- note: an OpenApiFeature must be configured in the CXFNonSpringJaxrsServlet -->
59 <dependency>
60 <groupId>org.apache.cxf</groupId>
61 <artifactId>cxf-rt-rs-service-description-openapi-v3</artifactId>
62 <version>4.1.1</version>
63 </dependency>
64 <dependency>
65 <groupId>org.webjars</groupId>
66 <artifactId>swagger-ui</artifactId>
67 <version>5.21.0</version>
68 </dependency>
69 <dependency>
70 <groupId>io.swagger.core.v3</groupId>
71 <artifactId>swagger-annotations</artifactId>
72 <version>2.2.30</version>
73 </dependency>
74 <!-- used for automatic conversion JSON2Object and Object2JSON -->
75 <!-- note: a JacksonJsonProvider must be configured in the CXFNonSpringJaxrsServlet -->
76 <dependency>
77 <groupId>com.fasterxml.jackson.iakarta.rs</groupId>
```

Annotations in the image:

- 1. Points to the first dependency block: `<groupId>org.apache.cxf</groupId>`, `<artifactId>cxf-rt-rs-service-description-openapi-v3</artifactId>`, and `<version>4.1.1</version>`. A note box says: "For generating OpenAPI v3.0 documents from JAX-RS service endpoints".
- 2. Points to the second dependency block: `<groupId>org.webjars</groupId>`, `<artifactId>swagger-ui</artifactId>`, and `<version>5.21.0</version>`. A note box says: "For including client-side web libraries (e.g., jQuery & Bootstrap) packaged into JAR".
- 3. Points to the third dependency block: `<groupId>io.swagger.core.v3</groupId>`, `<artifactId>swagger-annotations</artifactId>`, and `<version>2.2.30</version>`. A note box says: "Offers annotations to produce Swagger documentation".

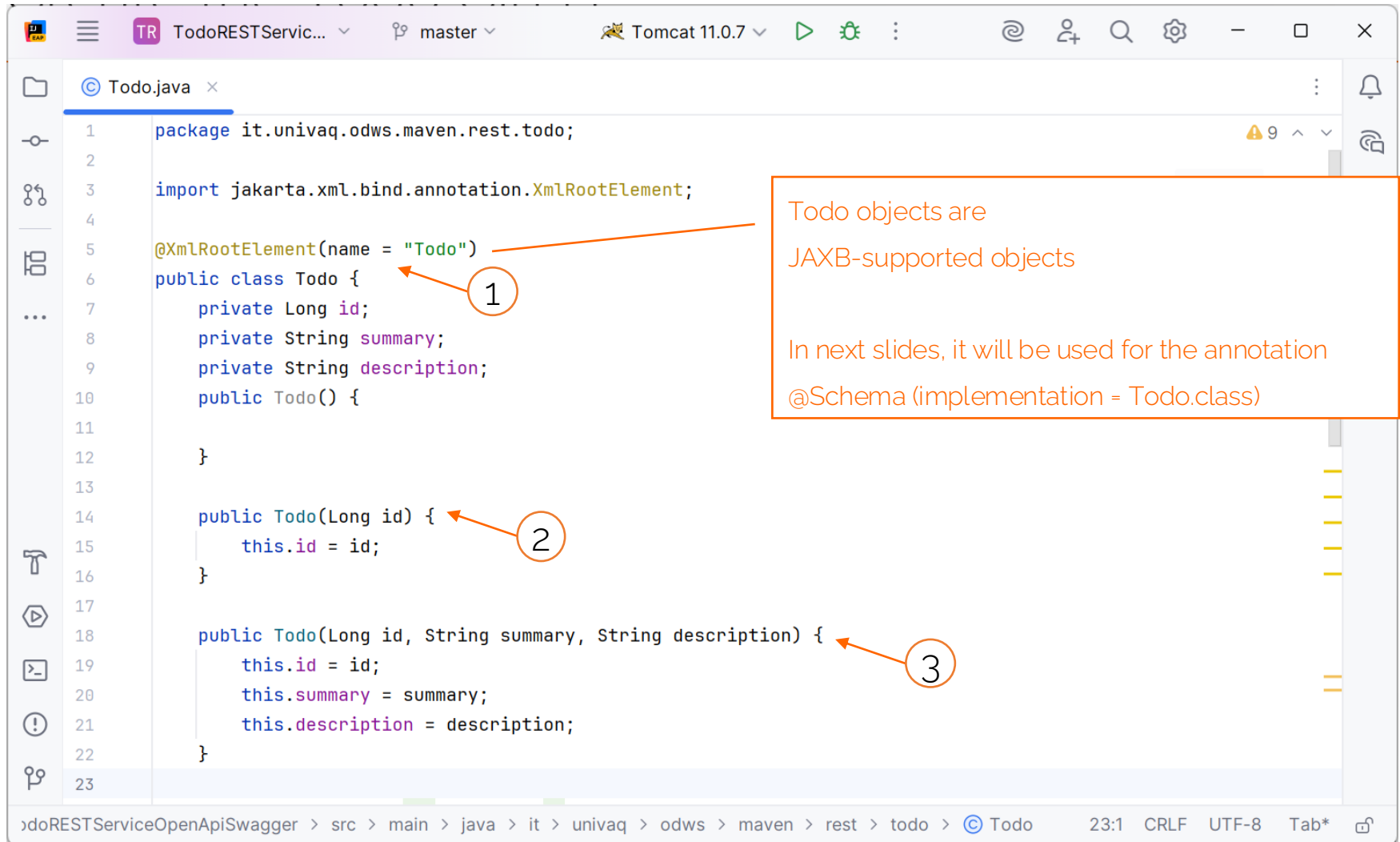
The bottom status bar shows "57:9 CRLF UTF-8 Tab*".

Add jackson-jakarta-rs-json provider dependency

Automatically serialize and deserialize resources for JSON content MediaType

```
1 <project xmlns="http://maven.apache.org/POM/4.0.0"
24 <dependencies>
69 <dependency>
73 </dependency>
74 <!-- used for automatic conversion JSON2Object and Object2JSON -->
75 <!-- note: a JacksonJsonProvider must be configured in the CXFNonSpringJaxrsServlet -->
76 <dependency>
    <groupId>com.fasterxml.jackson.jakarta.rs</groupId>
    <artifactId>jackson-jakarta-rs-json-provider</artifactId>
    <version>2.19.0</version>
80 </dependency>
81 </dependencies>
82
83 <build>
84 <plugins>
85 <plugin>
86 <groupId>org.apache.maven.plugins</groupId>
87 <artifactId>maven-compiler-plugin</artifactId>
88 <version>3.14.0</version>
89 <configuration>
90 <source>${maven.compiler.source}</source>
91 <target>${maven.compiler.target}</target>
92 <encoding>${project.build.sourceEncoding}</encoding>
```

Create the Todo Class



The screenshot shows an IDE window with the file `Todo.java` open. The code defines a `Todo` class with the following structure:

```
1 package it.univaq.odws.maven.rest.todo;
2
3 import jakarta.xml.bind.annotation.XmlRootElement;
4
5 @XmlRootElement(name = "Todo")
6 public class Todo {
7     private Long id;
8     private String summary;
9     private String description;
10    public Todo() {
11
12    }
13
14    public Todo(Long id) {
15        this.id = id;
16    }
17
18    public Todo(Long id, String summary, String description) {
19        this.id = id;
20        this.summary = summary;
21        this.description = description;
22    }
23 }
```

Annotations and constructors are highlighted with numbered circles and arrows:

- 1**: Points to the `@XmlRootElement(name = "Todo")` annotation.
- 2**: Points to the `public Todo(Long id)` constructor.
- 3**: Points to the `public Todo(Long id, String summary, String description)` constructor.

An orange box on the right contains the following text:

Todo objects are JAXB-supported objects

In next slides, it will be used for the annotation `@Schema (implementation = Todo.class)`

The bottom status bar shows the project path: `doRESTServiceOpenApiSwagger > src > main > java > it > univaq > odws > maven > rest > todo > © Todo`, along with settings like `23:1 CRLF UTF-8 Tab*`.

Annotation Type XmlRootElement

Example 1: Associate an element with XML Schema type

```
// Example: Code fragment
@XmlRootElement
class Point {
    int x;
    int y;
    Point(int _x,int _y) {x=_x;y=_y;}
}
```

```
//Example: Code fragment corresponding to XML output
marshal( new Point(3,5), System.out);
```

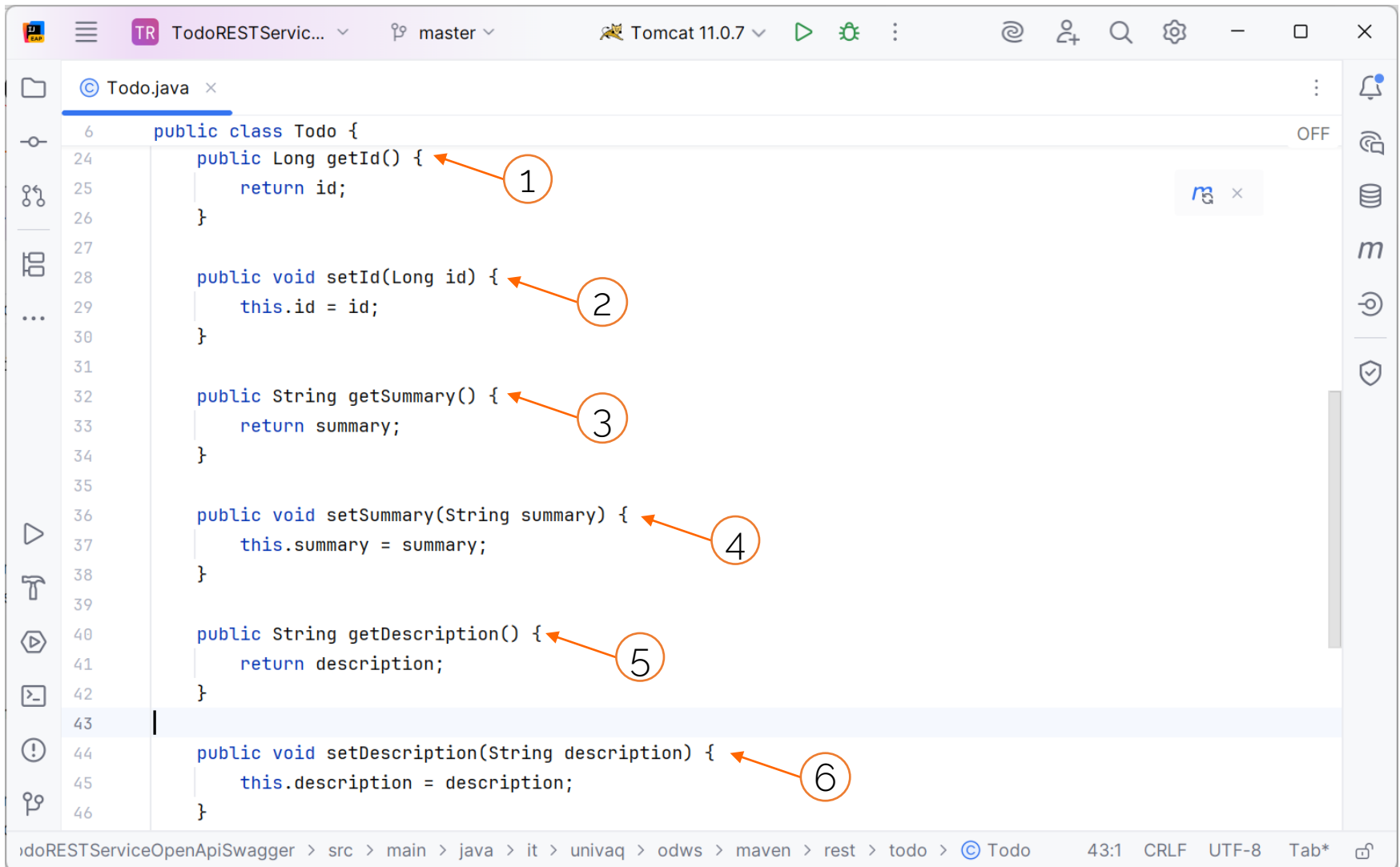
```
<!-- Example: XML output -->
<point>
  <x> 3 </x>
  <y> 5 </y>
</point>
```

The annotation causes an global element declaration to be produced in the schema. The global element declaration is associated with the XML schema type to which the class is mapped.

```
<!-- Example: XML schema definition -->
<xs:element name="point" type="point"/>
<xs:complexType name="point">
  <xs:sequence>
    <xs:element name="x" type="xs:int"/>
    <xs:element name="y" type="xs:int"/>
  </xs:sequence>
</xs:complexType>
```

<https://jakarta.ee/specifications/platform/9/apidocs/jakarta/xml/bind/annotation/xmlrootelement>

Create the Todo Class



The screenshot shows an IDE window for a project named 'TodoRESTService...'. The file 'Todo.java' is open, showing the following code:

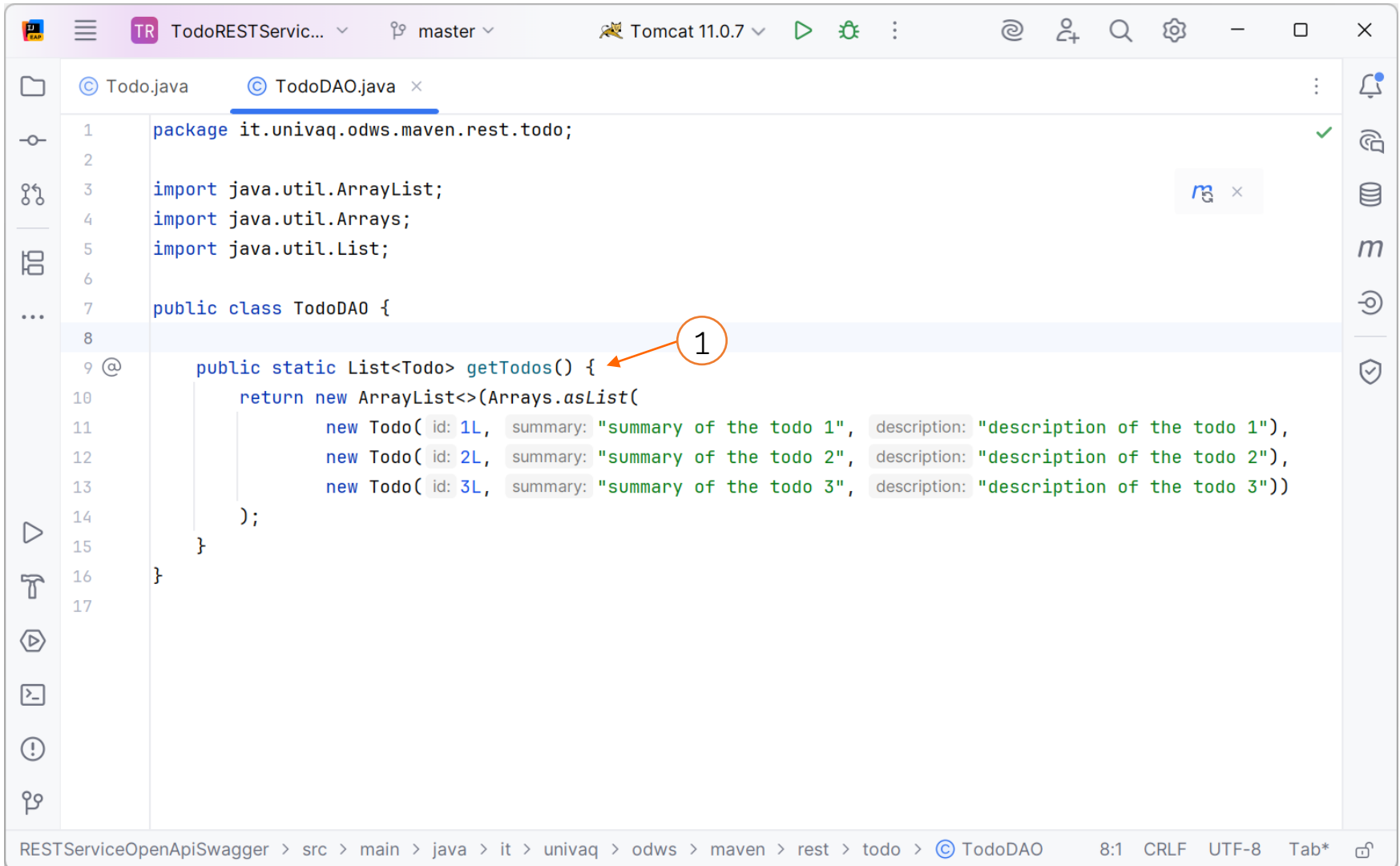
```
6 public class Todo {  
24     public Long getId() {  
25         return id;  
26     }  
27  
28     public void setId(Long id) {  
29         this.id = id;  
30     }  
31  
32     public String getSummary() {  
33         return summary;  
34     }  
35  
36     public void setSummary(String summary) {  
37         this.summary = summary;  
38     }  
39  
40     public String getDescription() {  
41         return description;  
42     }  
43  
44     public void setDescription(String description) {  
45         this.description = description;  
46     }
```

Numbered annotations (1-6) point to the following lines of code:

- 1: `public Long getId() {`
- 2: `public void setId(Long id) {`
- 3: `public String getSummary() {`
- 4: `public void setSummary(String summary) {`
- 5: `public String getDescription() {`
- 6: `public void setDescription(String description) {`

The IDE interface includes a sidebar with icons for file explorer, search, and other tools. The bottom status bar shows the file path: `TodoRESTServiceOpenApiSwagger > src > main > java > it > univaq > odws > maven > rest > todo > Todo`, along with encoding and tab settings: `43:1 CRLF UTF-8 Tab*`.

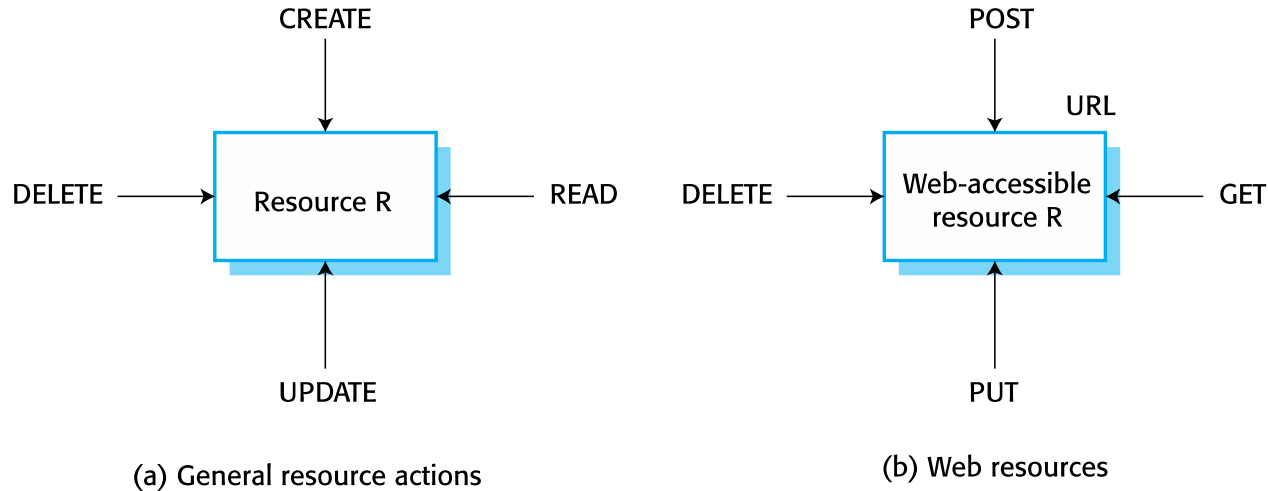
Create the TodoDAO class



```
1 package it.univaq.odws.maven.rest.todo;
2
3 import java.util.ArrayList;
4 import java.util.Arrays;
5 import java.util.List;
6
7 public class TodoDAO {
8
9     @ public static List<Todo> getTodos() {
10         return new ArrayList<>(Arrays.asList(
11             new Todo( id: 1L, summary: "summary of the todo 1", description: "description of the todo 1"),
12             new Todo( id: 2L, summary: "summary of the todo 2", description: "description of the todo 2"),
13             new Todo( id: 3L, summary: "summary of the todo 3", description: "description of the todo 3"))
14         );
15     }
16 }
17
```

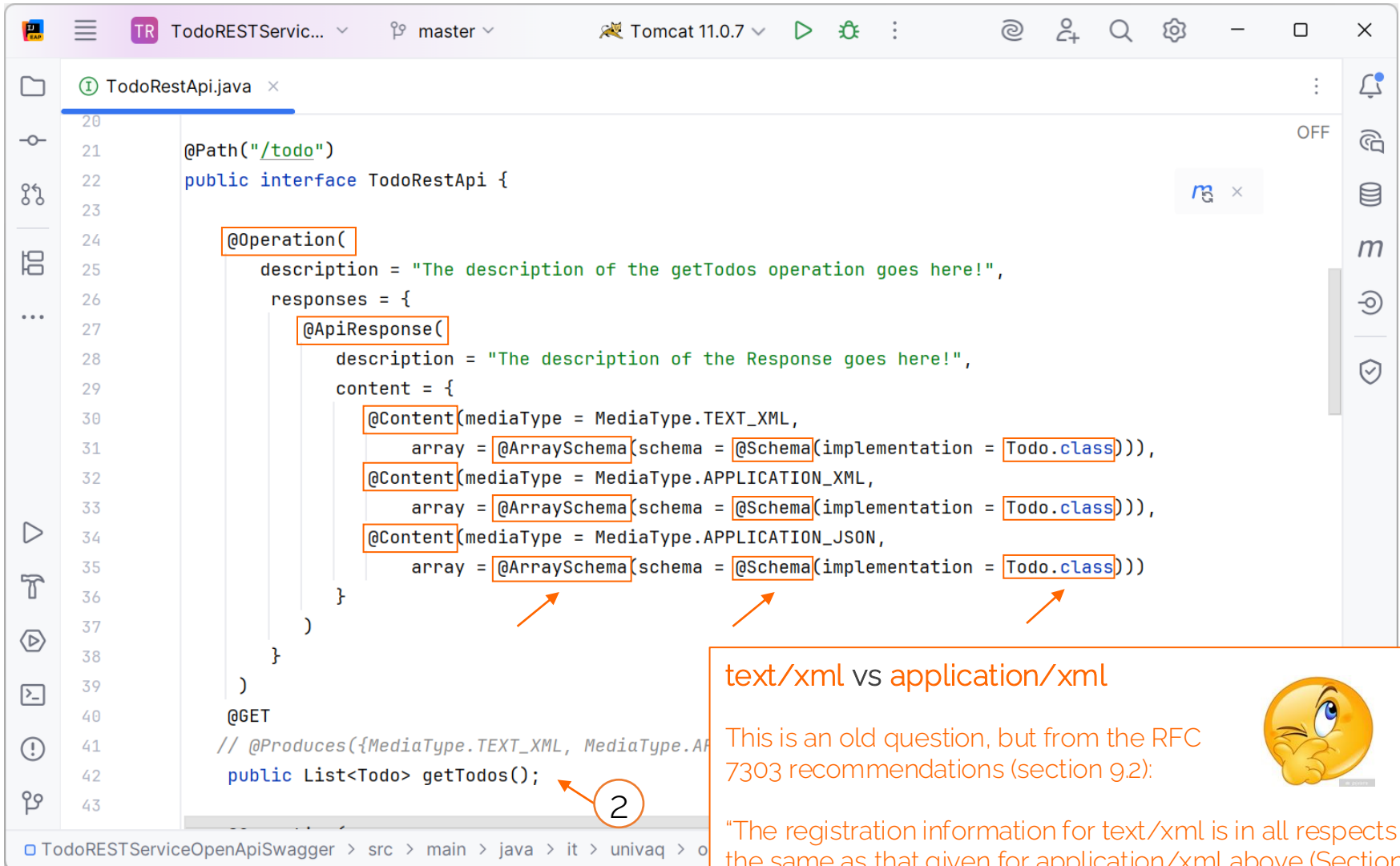
RESTServiceOpenApiSwagger > src > main > java > it > univaq > odws > maven > rest > todo > TodoDAO 8:1 CRLF UTF-8 Tab*

Just not to forget... 😊



- **POST** is used to **create** a resource. It has associated data that defines the resource
- **GET** is used to **read** the value of a resource and return that to the requestor in the specified representation, such as XHTML, that can be rendered in a web browser
- **PUT** is used to **update** the value of a resource
- **DELETE** is used to **delete** the resource

Create the TodoRestApi Interface



```
20
21 @Path("/todo")
22 public interface TodoRestApi {
23
24     @Operation(
25         description = "The description of the getTodos operation goes here!",
26         responses = {
27             @ApiResponse(
28                 description = "The description of the Response goes here!",
29                 content = {
30                     @Content(mediaType = MediaType.TEXT_XML,
31                         array = @ArraySchema(schema = @Schema(implementation = Todo.class))),
32                     @Content(mediaType = MediaType.APPLICATION_XML,
33                         array = @ArraySchema(schema = @Schema(implementation = Todo.class))),
34                     @Content(mediaType = MediaType.APPLICATION_JSON,
35                         array = @ArraySchema(schema = @Schema(implementation = Todo.class)))
36                 }
37             )
38         }
39     )
40     @GET
41     // @Produces({MediaType.TEXT_XML, MediaType.APPLICATION_XML})
42     public List<Todo> getTodos();
43 }
```

text/xml vs application/xml

This is an old question, but from the RFC 7303 recommendations (section 9.2):

“The registration information for text/xml is in all respects the same as that given for application/xml above (Section 9.1), except that the “Type name” is “text”.”

2

Create the TodoRestApi Interface

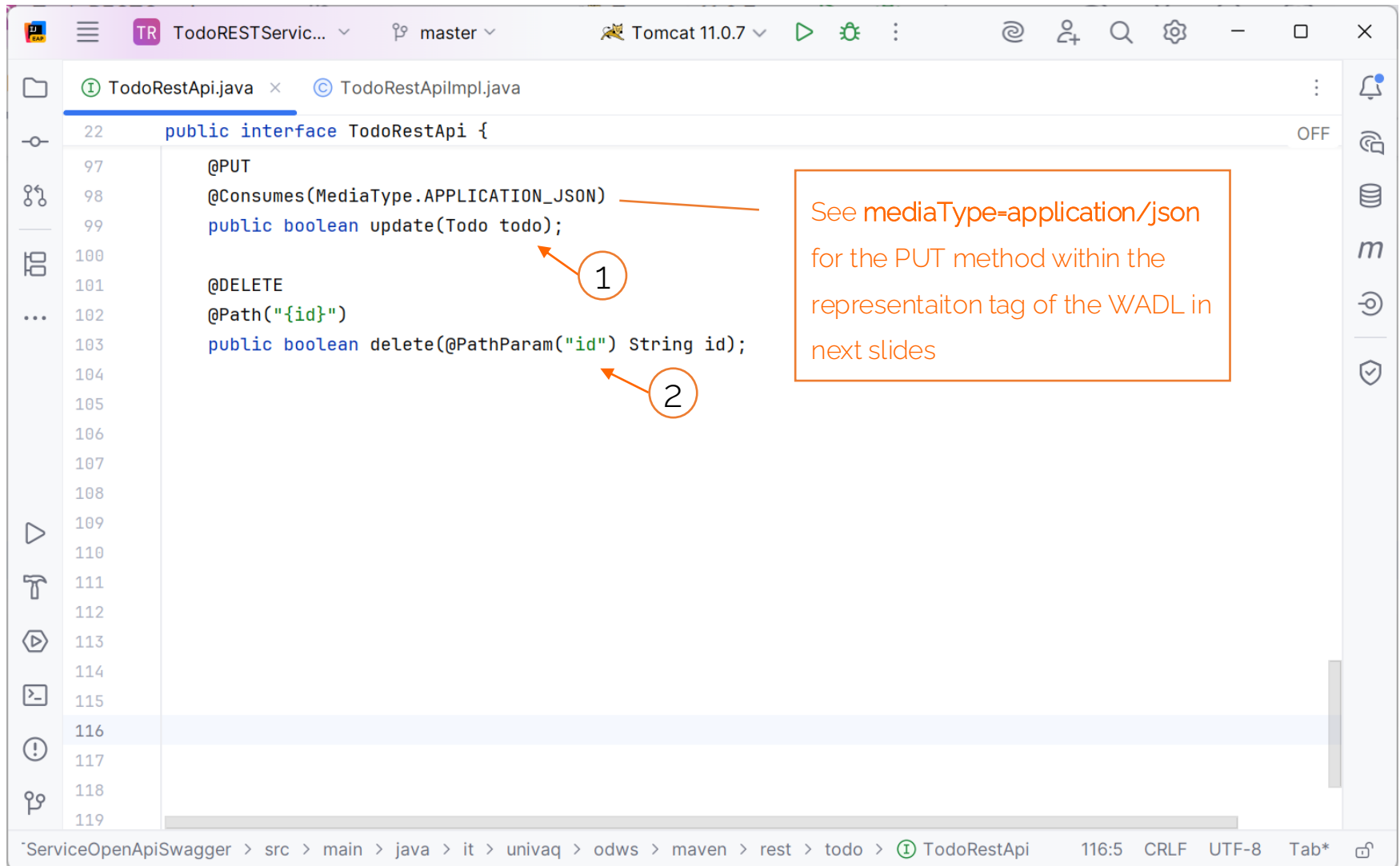
```
22 public interface TodoRestApi {  
44     @ApiOperation(  
45         description = "The description of the getTodo operation goes here!",  
46         responses = {  
47             @ApiResponse(  
48                 description = "The description of the Response goes here!",  
49                 content = @Content(mediaType = MediaType.APPLICATION_JSON,  
50                     schema = @Schema(implementation = Todo.class)))  
51             }  
52     )  
53     @GET  
54     @Path("/{id}")  
55     public Todo getTodo(@PathParam(value = "id") String id);  
56  
57  
58  
59  
60  
61  
62  
63  
64  
65  
66
```

Create the TodoRestApi Interface

```
22 public interface TodoRestApi {
59     @Operation(
60         description = "The description of the create operation goes here!",
61         responses = {
62             @ApiResponse(
63                 description = "The description of the Response goes here!",
64                 content = @Content(mediaType = MediaType.APPLICATION_JSON,
65                                 schema = @Schema(implementation = Boolean.class)))
66         }
67     )
68     @POST
69     public boolean create(
70         @RequestBody(description = "The description of the input parameter goes here!",
71                     required = true,
72                     content = {@Content(mediaType = MediaType.APPLICATION_XML,
73                                     schema = @Schema(implementation = Todo.class)),
74                                @Content(mediaType = MediaType.APPLICATION_JSON,
75                                     schema = @Schema(implementation = Todo.class)),
76                                }
77         ) Todo todo);
78
79
80
81
```

ApiSwagger > src > main > java > it > univaq > odws > maven > rest > todo > TodoRestApi > create 69:5 CRLF UTF-8 Tab*

Create the TodoRestApi Interface



The screenshot shows an IDE window with the file `TodoRestApi.java` open. The code defines a `public interface TodoRestApi` with two methods:

```
22 public interface TodoRestApi {  
97     @PUT  
98     @Consumes(MediaType.APPLICATION_JSON)  
99     public boolean update(Todo todo);  
100  
101     @DELETE  
102     @Path("/{id}")  
103     public boolean delete(@PathParam("id") String id);  
104  
105  
106  
107  
108  
109  
110  
111  
112  
113  
114  
115  
116  
117  
118  
119
```

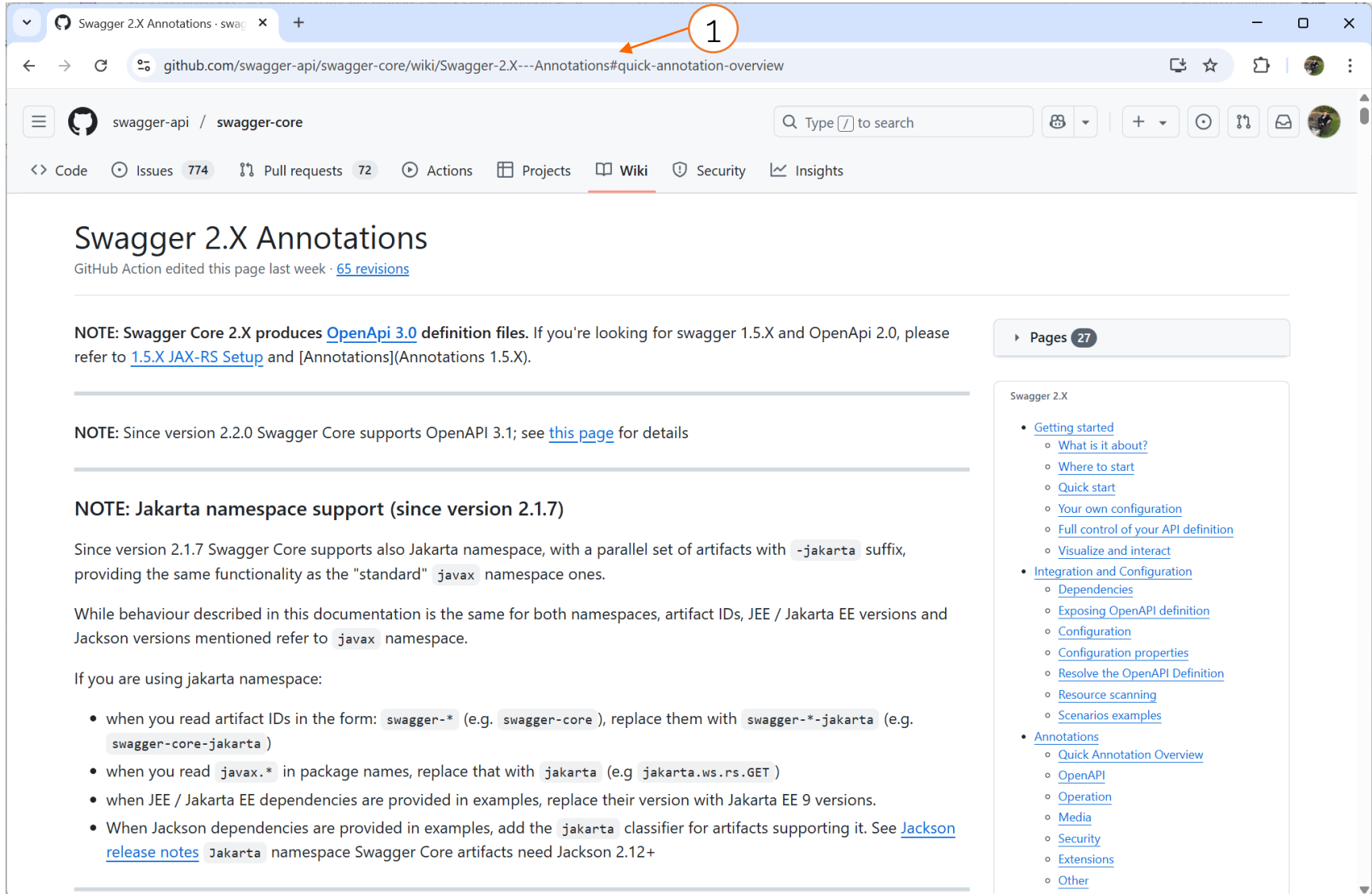
Annotations are highlighted with numbered circles:

- Circle 1 points to the `@PUT` annotation.
- Circle 2 points to the `@PathParam("id")` annotation.

A callout box on the right contains the text: "See `mediaType=application/json` for the PUT method within the representaiton tag of the WADL in next slides".

The IDE interface includes a top bar with project information (TR, master), a toolbar with icons for running, debugging, and searching, and a bottom status bar showing the current file path: `'ServiceOpenApiSwagger > src > main > java > it > univaq > odws > maven > rest > todo > TodoRestApi`.

Complete Description of Swagger Annotations



Swagger 2.X Annotations · swag x +

github.com/swagger-api/swagger-core/wiki/Swagger-2.X---Annotations#quick-annotation-overview

swagger-api / swagger-core

Code Issues 774 Pull requests 72 Actions Projects Wiki Security Insights

Swagger 2.X Annotations

GitHub Action edited this page last week · [65 revisions](#)

NOTE: Swagger Core 2.X produces [OpenApi 3.0](#) definition files. If you're looking for swagger 1.5.X and OpenApi 2.0, please refer to [1.5.X JAX-RS Setup](#) and [\[Annotations\]\(Annotations 1.5.X\)](#).

NOTE: Since version 2.2.0 Swagger Core supports OpenAPI 3.1; see [this page](#) for details

NOTE: Jakarta namespace support (since version 2.1.7)

Since version 2.1.7 Swagger Core supports also Jakarta namespace, with a parallel set of artifacts with `-jakarta` suffix, providing the same functionality as the "standard" `javax` namespace ones.

While behaviour described in this documentation is the same for both namespaces, artifact IDs, JEE / Jakarta EE versions and Jackson versions mentioned refer to `javax` namespace.

If you are using jakarta namespace:

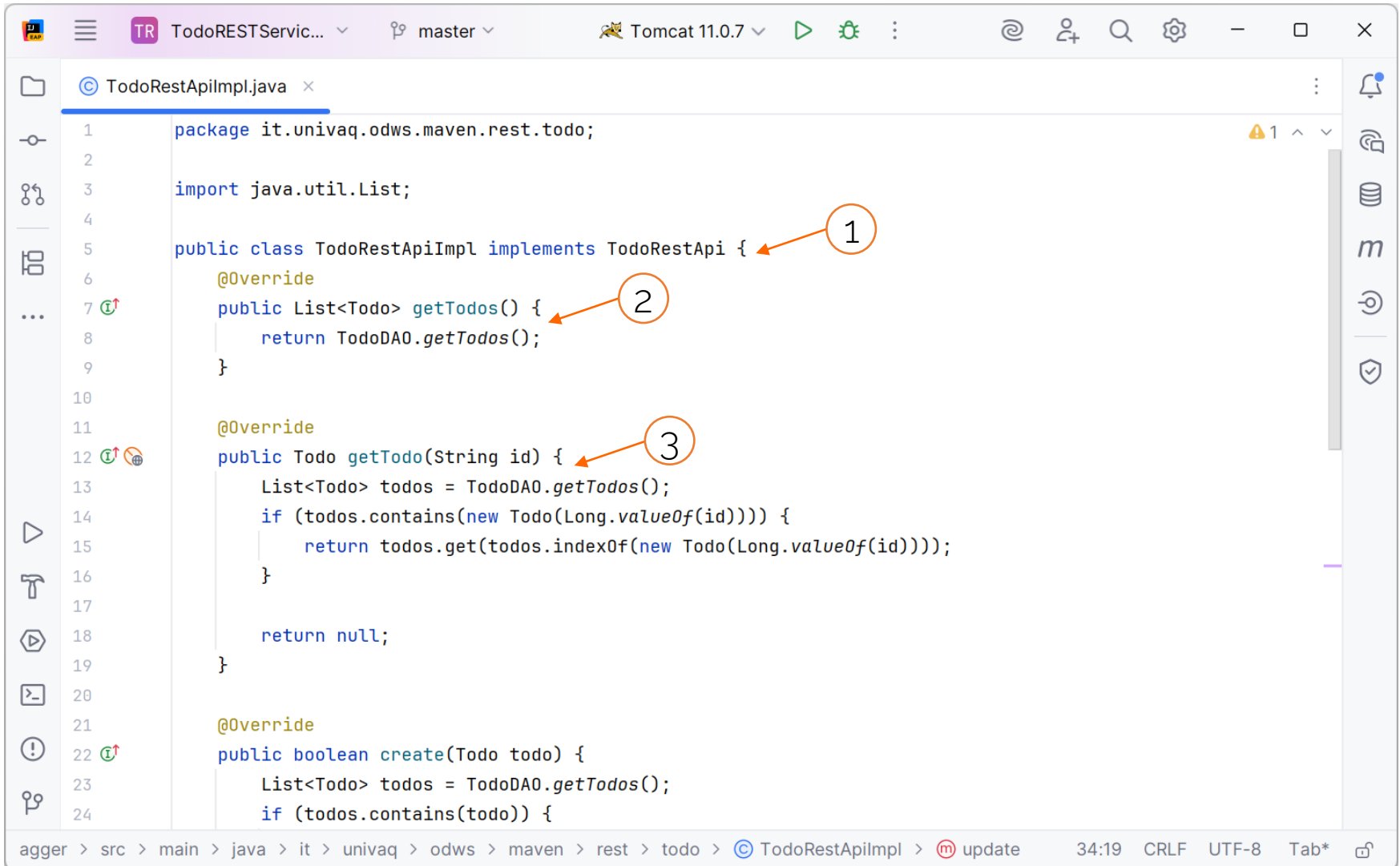
- when you read artifact IDs in the form: `swagger-*` (e.g. `swagger-core`), replace them with `swagger-*-jakarta` (e.g. `swagger-core-jakarta`)
- when you read `javax.*` in package names, replace that with `jakarta` (e.g. `jakarta.ws.rs.GET`)
- when JEE / Jakarta EE dependencies are provided in examples, replace their version with Jakarta EE 9 versions.
- When Jackson dependencies are provided in examples, add the `jakarta` classifier for artifacts supporting it. See [Jackson release notes](#) Jakarta namespace Swagger Core artifacts need Jackson 2.12+

Pages 27

Swagger 2.X

- Getting started
 - [What is it about?](#)
 - [Where to start](#)
 - [Quick start](#)
 - [Your own configuration](#)
 - [Full control of your API definition](#)
 - [Visualize and interact](#)
- Integration and Configuration
 - [Dependencies](#)
 - [Exposing OpenAPI definition](#)
 - [Configuration](#)
 - [Configuration properties](#)
 - [Resolve the OpenAPI Definition](#)
 - [Resource scanning](#)
 - [Scenarios examples](#)
- Annotations
 - [Quick Annotation Overview](#)
 - [OpenAPI](#)
 - [Operation](#)
 - [Media](#)
 - [Security](#)
 - [Extensions](#)
 - [Other](#)

Implement the TodoRestApi Interface



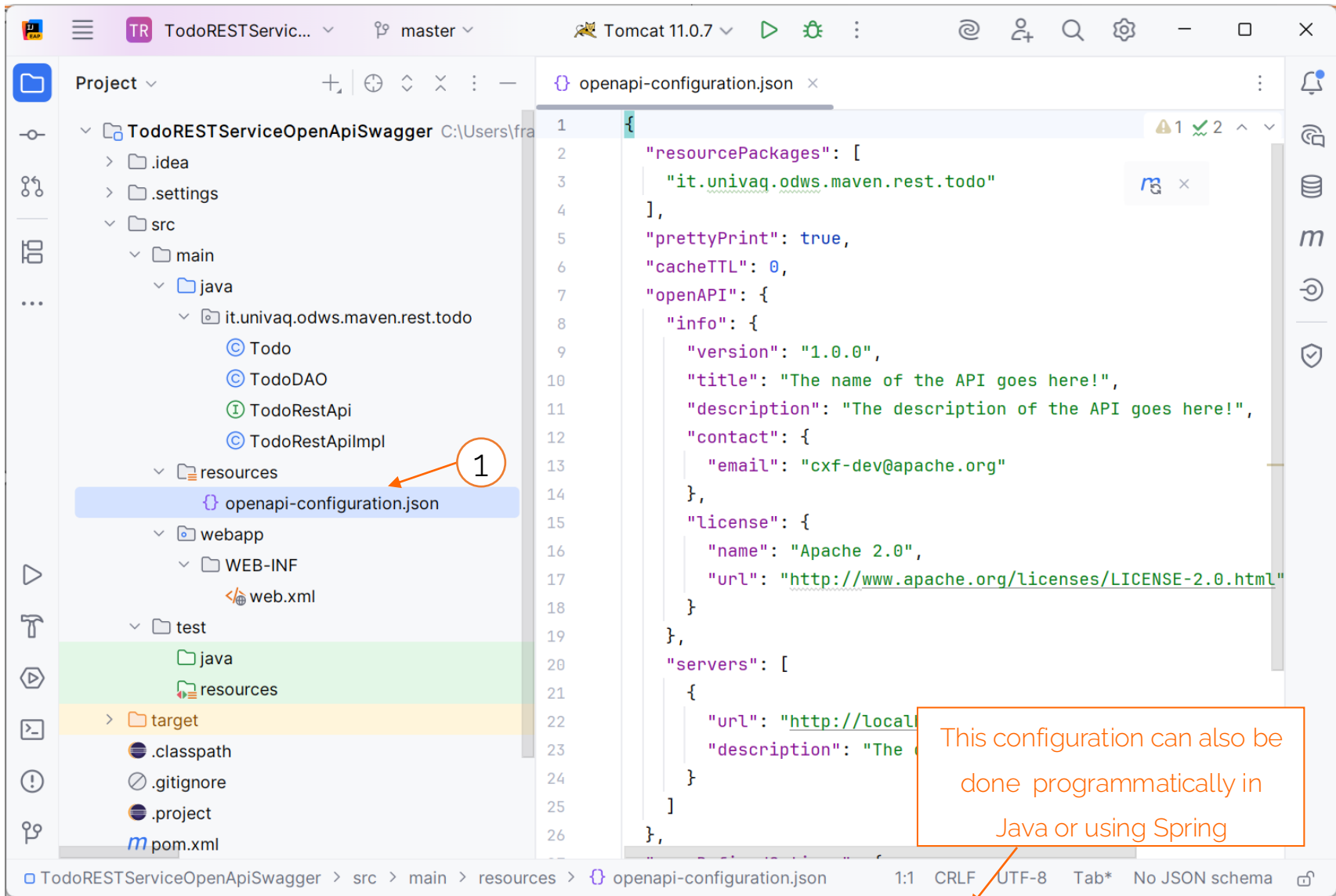
```
1 package it.univaq.odws.maven.rest.todo;
2
3 import java.util.List;
4
5 public class TodoRestApiImpl implements TodoRestApi {
6     @Override
7     public List<Todo> getTodos() {
8         return TodoDAO.getTodos();
9     }
10
11     @Override
12     public Todo getTodo(String id) {
13         List<Todo> todos = TodoDAO.getTodos();
14         if (todos.contains(new Todo(Long.valueOf(id)))) {
15             return todos.get(todos.indexOf(new Todo(Long.valueOf(id))));
16         }
17
18         return null;
19     }
20
21     @Override
22     public boolean create(Todo todo) {
23         List<Todo> todos = TodoDAO.getTodos();
24         if (todos.contains(todo)) {
```

Implement the TodoRestApi Interface

```
5 public class TodoRestApiImpl implements TodoRestApi {
21     @Override
22     public boolean create(Todo todo) {
23         List<Todo> todos = TodoDAO.getTodos();
24         if (todos.contains(todo)) {
25             return false;
26         }
27         return todos.add(todo);
28     }
29
30     @Override
31     public boolean update(Todo todo) {
32         List<Todo> todos = TodoDAO.getTodos();
33         if (todos.contains(todo)) {
34             todos.remove(todos.indexOf(todo));
35         }
36         return todos.add(todo);
37     }
38     @Override
39     public boolean delete(String id) {
40         List<Todo> todos = TodoDAO.getTodos();
41         if (todos.contains(new Todo(Long.valueOf(id)))){
42             todos.remove(new Todo(Long.valueOf(id)));
43             return true;
44         }
45         return false;
46     }
47 }
```

The screenshot shows an IDE window titled 'TodoRESTService...' with a tab for 'TodoRestApiImpl.java'. The code implements the 'TodoRestApi' interface. Three annotations are present: '1' points to the 'create' method signature, '2' points to the 'update' method signature, and '3' points to the 'delete' method signature. The code uses 'TodoDAO' for database operations and 'Todo' objects for data representation. The IDE interface includes a sidebar with icons for Explorer, Search, Run and Debug, and other tools, and a bottom status bar showing the file path and encoding.

Create the openapi-configuration.json



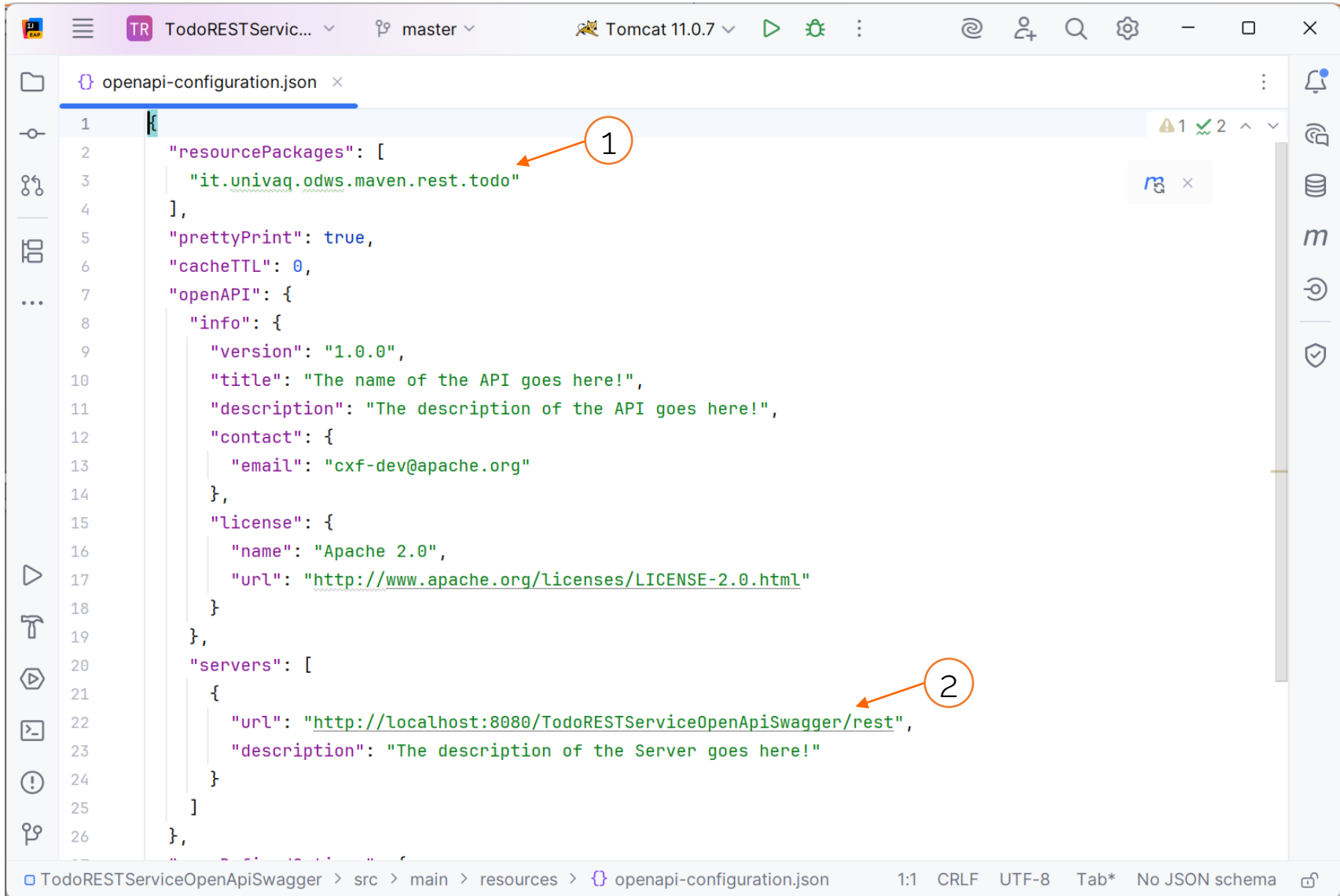
The screenshot shows an IDE window for a project named 'TodoRESTServiceOpenApiSwagger'. The left sidebar displays the project structure, with the file 'openapi-configuration.json' in the 'resources' directory highlighted and circled with an orange '1'. The main editor displays the JSON configuration for the OpenAPI specification:

```
1 {
2   "resourcePackages": [
3     "it.univaq.odws.maven.rest.todo"
4   ],
5   "prettyPrint": true,
6   "cacheTTL": 0,
7   "openAPI": {
8     "info": {
9       "version": "1.0.0",
10      "title": "The name of the API goes here!",
11      "description": "The description of the API goes here!",
12      "contact": {
13        "email": "cxf-dev@apache.org"
14      },
15      "license": {
16        "name": "Apache 2.0",
17        "url": "http://www.apache.org/licenses/LICENSE-2.0.html"
18      }
19    },
20    "servers": [
21      {
22        "url": "http://localhost",
23        "description": "The"
24      }
25    ]
26  }
```

An orange callout box with an arrow pointing to the 'servers' section contains the text: "This configuration can also be done programmatically in Java or using Spring".

<https://cwiki.apache.org/confluence/display/CXF20DOC/OpenApiFeature#OpenApiFeature-ConfiguringinCXFNonSpringJaxrsServlet>

Create the openapi-configuration.json



The screenshot shows an IDE window with the file `openapi-configuration.json` open. The file content is as follows:

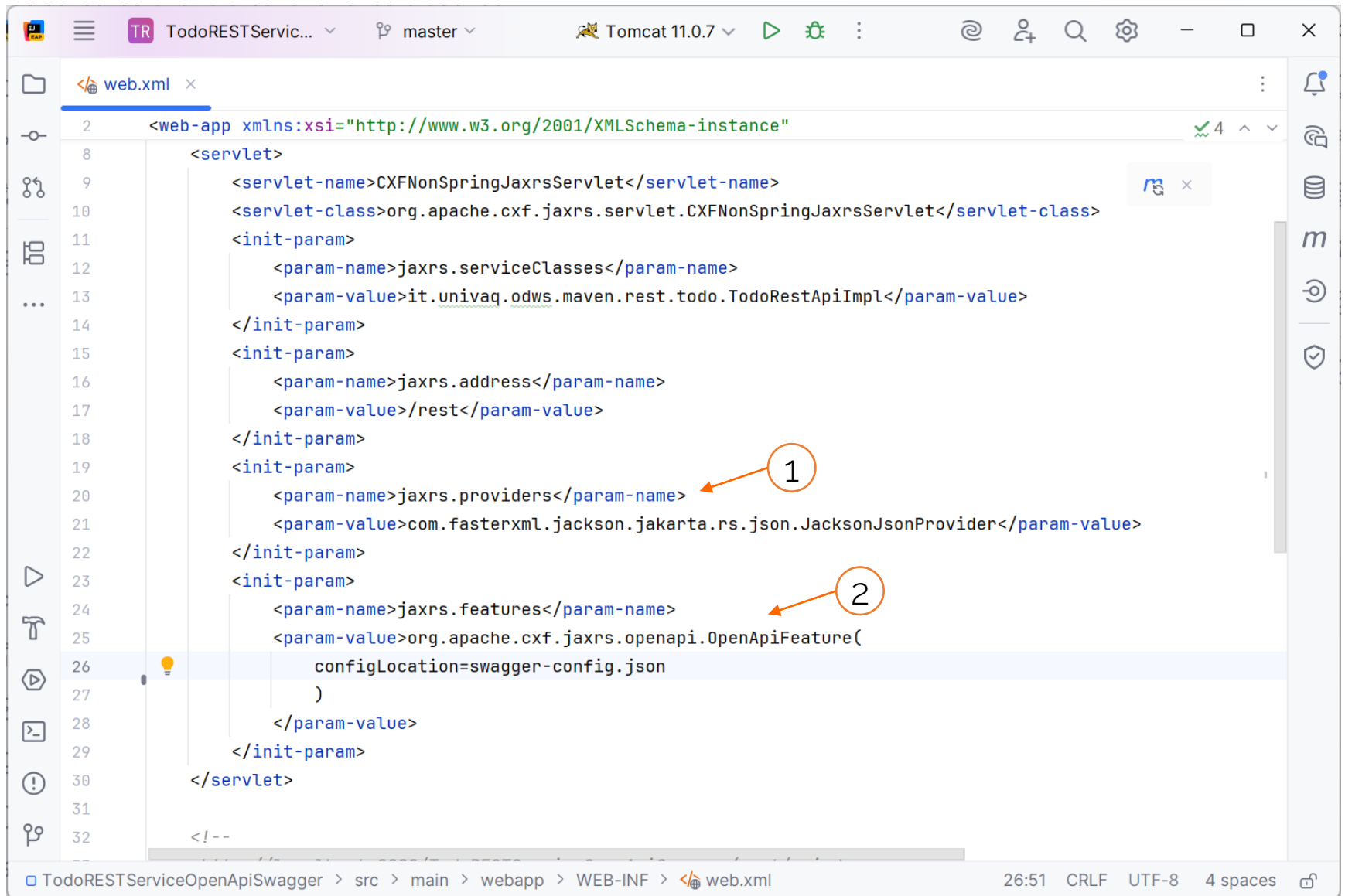
```
1  {
2    "resourcePackages": [
3      "it.univaq.odws.maven.rest.todo"
4    ],
5    "prettyPrint": true,
6    "cacheTTL": 0,
7    "openAPI": {
8      "info": {
9        "version": "1.0.0",
10       "title": "The name of the API goes here!",
11       "description": "The description of the API goes here!",
12       "contact": {
13         "email": "cxf-dev@apache.org"
14       },
15       "license": {
16         "name": "Apache 2.0",
17         "url": "http://www.apache.org/licenses/LICENSE-2.0.html"
18       }
19     },
20    "servers": [
21      {
22        "url": "http://localhost:8080/ToDoRESTServiceOpenApiSwagger/rest",
23        "description": "The description of the Server goes here!"
24      }
25    ]
26  },
```

Two annotations are present:

- An arrow labeled **1** points to the `"resourcePackages"` array on line 2.
- An arrow labeled **2** points to the `"url"` property of the first server in the `"servers"` array on line 22.

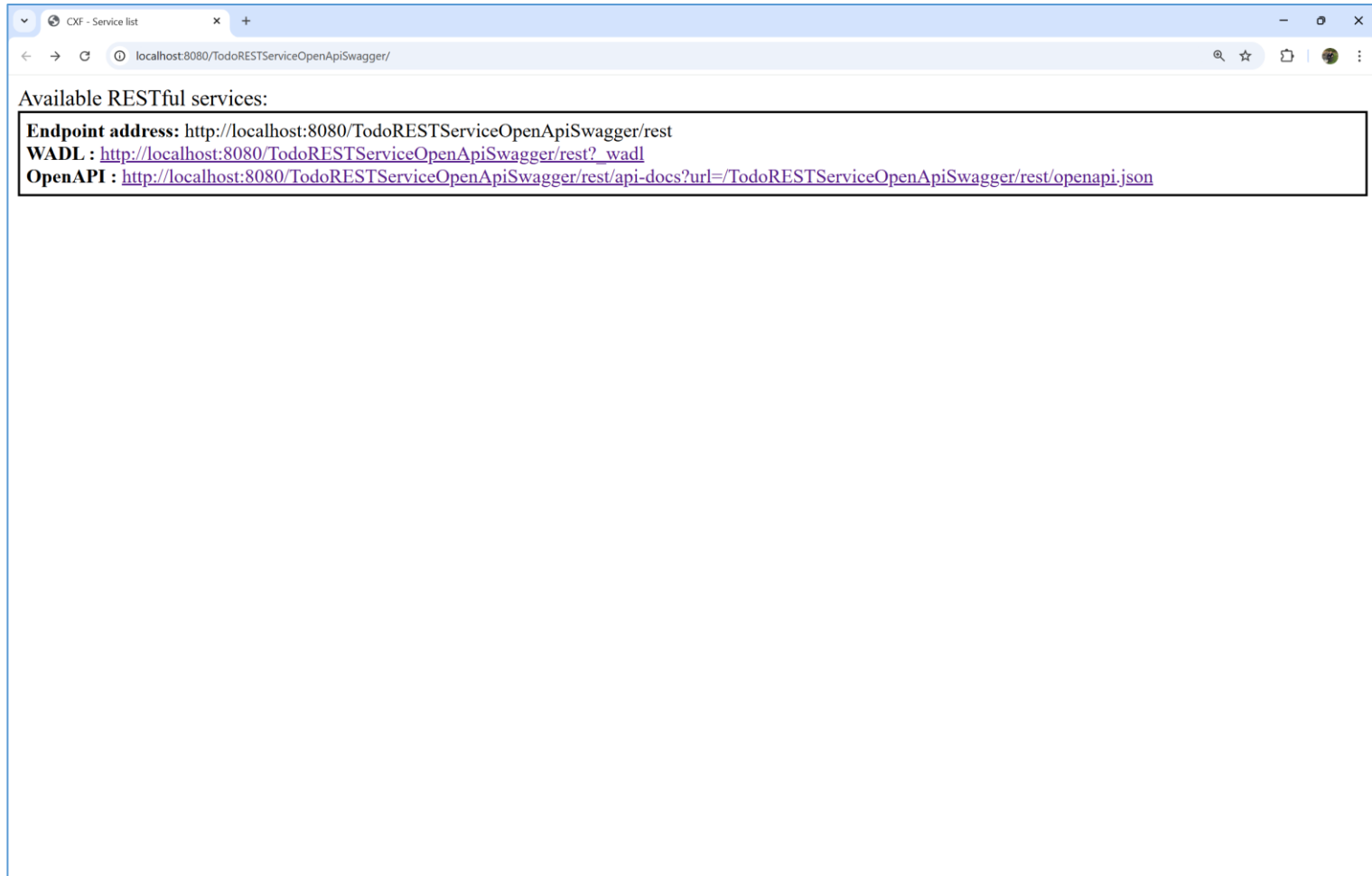
The IDE interface includes a top bar with project and server information, a left sidebar with navigation icons, and a bottom status bar showing the file path and encoding details.

Configure the servlet



```
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  <servlet>
    <servlet-name>CXFNonSpringJaxrsServlet</servlet-name>
    <servlet-class>org.apache.cxf.jaxrs.servlet.CXFNonSpringJaxrsServlet</servlet-class>
    <init-param>
      <param-name>jaxrs.serviceClasses</param-name>
      <param-value>it.univaq.odws.maven.rest.todo.TODORestApiImpl</param-value>
    </init-param>
    <init-param>
      <param-name>jaxrs.address</param-name>
      <param-value>/rest</param-value>
    </init-param>
    <init-param>
      <param-name>jaxrs.providers</param-name>
      <param-value>com.fasterxml.jackson.jakarta.rs.json.JacksonJsonProvider</param-value>
    </init-param>
    <init-param>
      <param-name>jaxrs.features</param-name>
      <param-value>org.apache.cxf.jaxrs.openapi.OpenApiFeature(
        configLocation=swagger-config.json
      )
    </param-value>
    </init-param>
  </servlet>
  <!--
```

Deploy the REST Service



The REST API description - WADL



This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
<application xmlns="http://wadl.dev.java.net/2009/02" xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <grammars>
    <xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema" attributeFormDefault="unqualified"
      elementFormDefault="unqualified" version="1.0">
      <xs:element name="Todo" type="todo"/>
      <xs:complexType name="todo">
        <xs:sequence>
          <xs:element minOccurs="0" name="description" type="xs:string"/>
          <xs:element minOccurs="0" name="id" type="xs:long"/>
          <xs:element minOccurs="0" name="summary" type="xs:string"/>
        </xs:sequence>
      </xs:complexType>
    </xs:schema>
  </grammars>
  <resources base="http://localhost:8080/ToDoRESTServiceOpenApiSwagger/rest">
    <resource path="/todo"> ← 1
      ...
    </resource>
    <resource path="/openapi.{type:json|yaml}"> ← 2
      ...
    </resource>
    <resource path="/api-docs"> ← 3
      ...
    </resource>
  </resources>
</application>
```

The REST API description - WADL

```
</grammars>
▼ <resources base="http://localhost:8080/ToDoRESTServiceOpenApiSwagger/rest">
  ▼ <resource path="/todo">
    ▼ <method name="GET"> ← 1.1
      ▼ <response>
        <representation mediaType="*/*/">
          </representation>
        </response>
      </method>
    ▼ <method name="POST"> ← 1.2
      ▼ <request>
        <representation mediaType="*/*/">
          </representation>
        </request>
      ▼ <response>
        ▼ <representation mediaType="*/*/">
          <param name="result" style="plain" type="xs:boolean"/>
        </representation>
      </response>
    </method>
    ▼ <method name="PUT"> ← 1.3
      ▼ <request>
        <representation mediaType="application/json">
          </representation>
        </request>
      ▼ <response>
        ▼ <representation mediaType="*/*/">
          <param name="result" style="plain" type="xs:boolean"/>
        </representation>
      </response>
    </method>
    ▼ <resource path="/{id}">
      <param name="id" style="template" type="xs:string"/>
      ▼ <method name="DELETE">
        <request/>
      </method>
    </resource>
  </resource>
</resources>
```

The REST Api description - WADL

```
<xs:element minOccurs="0" name="summary" type="xs:string"/>
</xs:sequence>
</xs:complexType>
</xs:schema>
</grammars>
▼ <resources base="http://localhost:8080/ToDoRESTServiceOpenApiSwagger/rest">
  ▶ <resource path="/todo">
    ...
  </resource>
  ▼ <resource path="/openapi.{type:json|yaml}"> ← 2
    <param name="type" style="template" type="xs:string"/>
    ▼ <method name="GET"> ← 2.1
      <request/>
      ▼ <response>
        <representation mediaType="application/json"/>
        <representation mediaType="application/yaml"/>
      </response>
    </method>
  </resource>
  ▼ <resource path="/api-docs"> ← 3
    ▼ <resource path="{resource:.*)">
      <param name="resource" style="template" type="xs:string"/>
      ▼ <method name="GET"> ← 3.1
        <request/>
        ▼ <response>
          <representation mediaType="*/*/>
        </response>
      </method>
    </resource>
  </resource>
</resources>
</application>
```

The REST API description - JSON

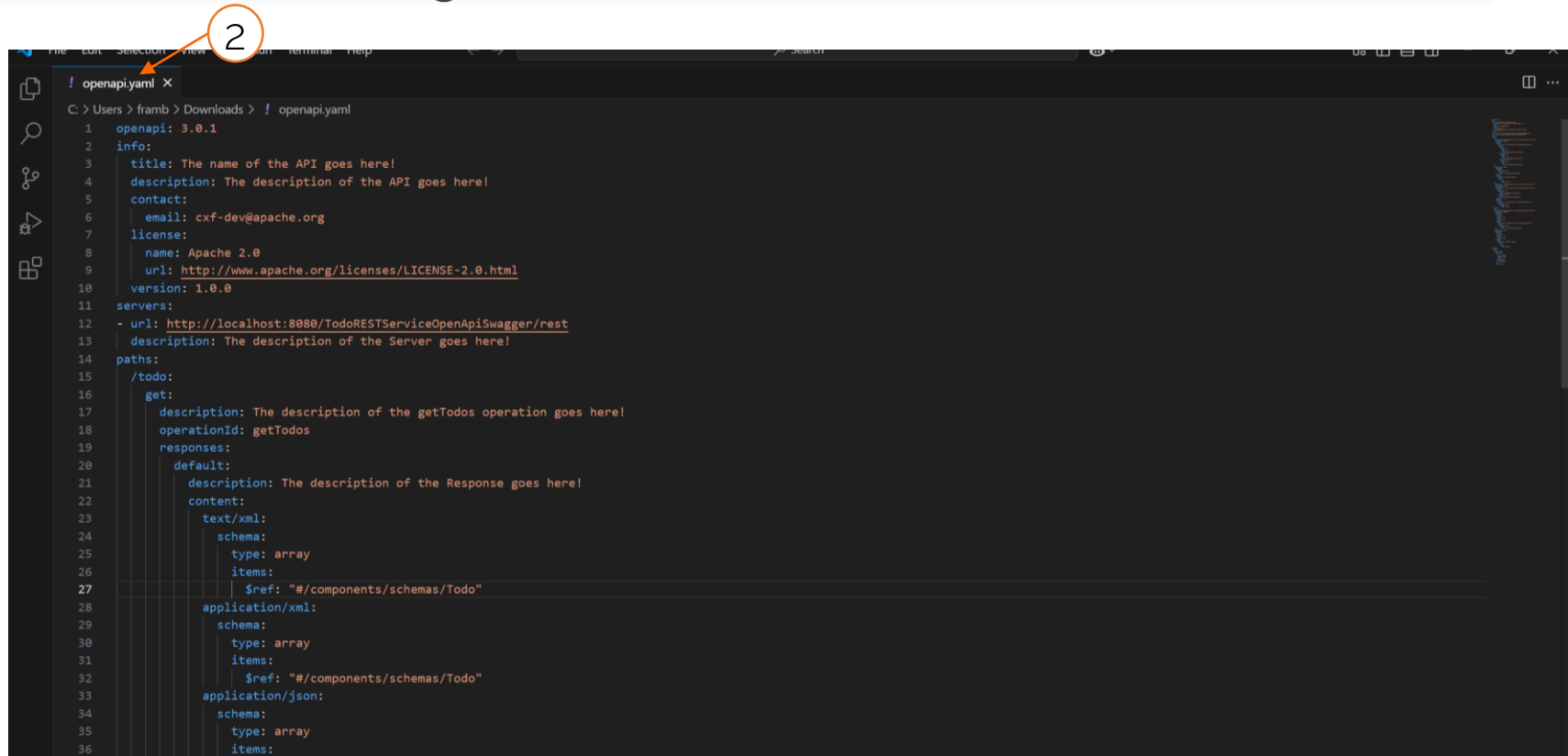
localhost:8080/ToDoRESTService x +

localhost:8080/ToDoRESTServiceOpenApiSwagger/rest/openapi.json

Formatta il codice ☐

```
{
  "openapi" : "3.0.1",
  "info" : {
    "title" : "The name of the API goes here!",
    "description" : "The description of the API goes here!",
    "contact" : {
      "email" : "cxf-dev@apache.org"
    },
    "license" : {
      "name" : "Apache 2.0",
      "url" : "http://www.apache.org/licenses/LICENSE-2.0.html"
    },
    "version" : "1.0.0"
  },
  "servers" : [ {
    "url" : "http://localhost:8080/ToDoRESTServiceOpenApiSwagger/rest",
    "description" : "The description of the Server goes here!"
  } ],
  "paths" : {
    "/todo" : {
      "get" : {
        "description" : "The description of the getTodos operation goes here!",
        "operationId" : "getTodos",
        "responses" : {
          "default" : {
            "description" : "The description of the Response goes here!",
            "content" : {
              "text/xml" : {
                "schema" : {
                  "type" : "array",
                  "items" : {
                    "$ref" : "#/components/schemas/ToDo"
                  }
                }
              }
            }
          }
        },
        "application/xml" : {
          "schema" : {
            "type" : "array",
            "items" : {
```

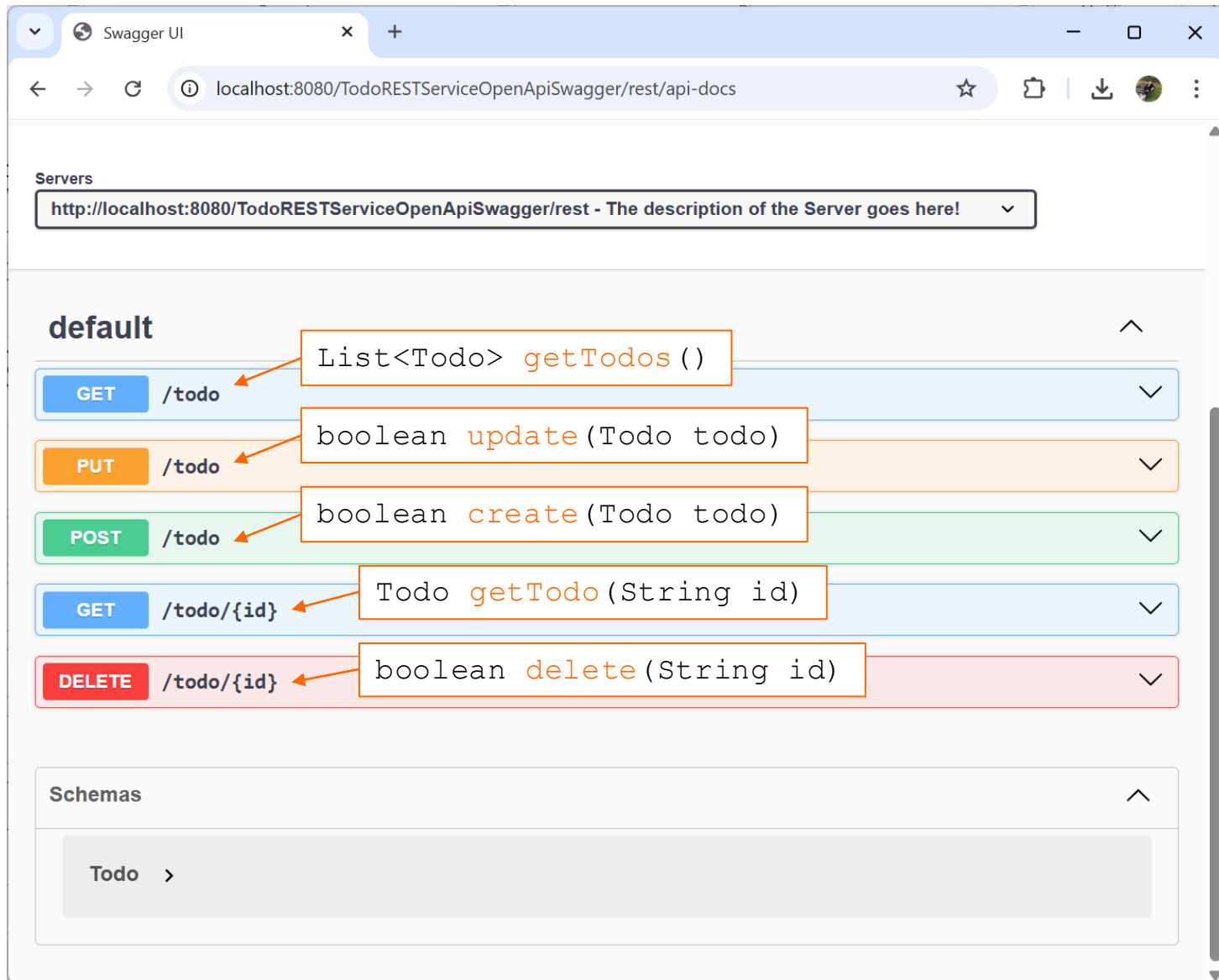
The REST Api description - YAML



Swagger api-docs

The screenshot shows the Swagger UI interface in a web browser. The browser's address bar is labeled with a circled '1' and points to the URL `localhost:8080/ToDoRESTServiceOpenApiSwagger/rest/api-docs`. The Swagger UI header features the Swagger logo, a text input field containing `/ToDoRESTServiceOpenApiSwagger/rest/openapi.json` (annotated with a circled '2' and a note 'point to .yaml or .json url'), and an 'Explore' button. The main content area displays the text 'The name of the API goes here!' followed by version tags '1.0.0' and 'OAS 3.0'. Below this is a 'Servers' section with a dropdown menu showing `http://localhost:8080/ToDoRESTServiceOpenApiSwagger/rest - The description of the Server goes here!`. The 'default' section lists three API endpoints: `GET /todo`, `PUT /todo`, and `POST /todo`, each with a corresponding colored button and a dropdown arrow. A circled '3' with an arrow points to the 'default' section header.

Swagger api-docs



Try to invoke GET /todo

Swagger UI

localhost:8080/TodoRESTServiceOpenApiSwagger/rest/api-docs#/default/getTodos

GET /todo

The description of the getTodos operation goes here!

Parameters

No parameters

Responses

Code **Description** **Links**

default The description of the Response goes here!

Media type

text/xml

text/xml

application/xml

application/json

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- XML example cannot be generated; root element name is undefined -->
```

Annotations:

- 1. `@ApiOperation(description = "The description of the getTodos operation goes here!", responses = { @ApiResponse(description = "The description of the Response goes here!", content = { @Content(mediaType = MediaType.TEXT_XML, array = @ArraySchema(schema = @Schema(implementation = Todo.class))), @Content(mediaType = MediaType.APPLICATION_XML, array = @ArraySchema(schema = @Schema(implementation = Todo.class))), @Content(mediaType = MediaType.APPLICATION_JSON, array = @ArraySchema(schema = @Schema(implementation = Todo.class))) }) })`
- 2. The description of the Response goes here!
- 3. Media type

Try to invoke GET /todo

Swagger UI

localhost:8080/ToDoRESTServiceOpenApiSwagger/rest/api-docs#/default/getTodos

GET /todo

The description of the getTodos operation goes here!

Parameters

No parameters

Try it out

Execute

Responses

Code	Description	Links
default	The description of the Response goes here! Media type text/xml Controls Accept header. Example Value Schema <pre><?xml version="1.0" encoding="UTF-8"?> <!-- XML example cannot be generated; root element name is undefined --></pre>	No links

Try to invoke GET /todo

The screenshot shows the Swagger UI interface in a web browser. The browser's address bar displays the URL: `localhost:8080/ToDoRESTServiceOpenApiSwagger/rest/api-docs?url=/ToDoRESTServiceOpenApiSwagger/rest/openapi.json#/d...`. The page title is "Swagger UI".

The "Responses" section is expanded, showing the details for the `GET /todo` endpoint. A red circle with the number "1" points to the "Curl" field, which contains the following command:

```
curl -X 'GET' \
  'http://localhost:8080/ToDoRESTServiceOpenApiSwagger/rest/todo' \
  -H 'accept: text/xml'
```

Below the curl command, the "Request URL" field shows: `http://localhost:8080/ToDoRESTServiceOpenApiSwagger/rest/todo`.

The "Server response" section shows a `200` status code. The "Response body" field contains the following XML:

```
<?xml version="1.0" encoding="UTF-8"?>
<Todos>
  <Todo>
    <description>description of the todo 1</description>
    <id>1</id>
    <summary>summary of the todo 1</summary>
  </Todo>
  <Todo>
    <description>description of the todo 2</description>
    <id>2</id>
    <summary>summary of the todo 2</summary>
  </Todo>
  <Todo>
    <description>description of the todo 3</description>
    <id>3</id>
    <summary>summary of the todo 3</summary>
  </Todo>
</Todos>
```

A red circle with the number "2" points to the first `<Todo>` element in the XML. A yellow callout box with the text "XML response body" is positioned next to the XML content.

The "Response headers" section shows the following headers:

```
connection: keep-alive
content-length: 398
content-type: text/xml
date: Fri, 23 May 2025 08:41:31 GMT
keep-alive: timeout=20
```

Try to invoke GET /todo

Swagger UI

localhost:8080/TodoRESTServiceOpenApiSwagger/rest/api-docs?url=/TodoRESTServiceOpenApiSwagger/rest/openapi.json#/default/getTodos

Curl

```
curl -X 'GET' \
  'http://localhost:8080/TodoRESTServiceOpenApiSwagger/rest/todo' \
  -H 'accept: application/json'
```

Request URL

http://localhost:8080/TodoRESTServiceOpenApiSwagger/rest/todo

Server response

Code 200 **Details**

Response body

```
[
  {
    "id": 1,
    "summary": "summary of the todo 1",
    "description": "description of the todo 1"
  },
  {
    "id": 2,
    "summary": "summary of the todo 2",
    "description": "description of the todo 2"
  },
  {
    "id": 3,
    "summary": "summary of the todo 3",
    "description": "description of the todo 3"
  }
]
```

Response headers

```
connection: keep-alive
content-type: application/json
date: Fri, 23 May 2025 08:43:25 GMT
keep-alive: timeout=20
transfer-encoding: chunked
```

Responses

Code	Description	Links
default	The description of the Response goes here!	No links

Media type:

Controls Accept header.

JSON response body

Try to invoke POST /todo

POST /todo

The description of the create operation goes here!

boolean create(Todo todo)

Parameters

No parameters

Request body **required**

The description of the input parameter goes here!

Example Value | Schema

```
<?xml version="1.0" encoding="UTF-8"?>
<Todo>
  <id>0</id>
  <summary>string</summary>
  <description>string</description>
</Todo>
```

Responses

Code	Description	Links
default	The description of the Response goes here!	No links

Media type

application/json

Controls Accept header.

Example Value | Schema

true

Try to invoke POST /todo

The screenshot shows the Swagger UI for a POST /todo endpoint. The interface is divided into several sections:

- Parameters:** A section with a "Try it out" button and the text "No parameters".
- Request body:** A section with a "required" label and a dropdown menu set to "application/xml". Below this is a text area with the description "The description of the input parameter goes here!".
- Example Value:** A section with a "Schema" link and a code editor containing XML:

```
<?xml version="1.0" encoding="UTF-8"?>
<Todo>
  <id>0</id>
  <summary>string</summary>
  <description>string</description>
</Todo>
```

 A yellow box labeled "XML request body" is overlaid on this section.
- Responses:** A section with a table of responses. The table has columns for "Code", "Description", and "Links". The "default" response has a description "The description of the Response goes here!". A yellow box labeled "1" points to the "Responses" section header.
- Media type:** A dropdown menu set to "application/json". A yellow box labeled "2" points to the "Description" field.
- Example Value:** A section with a "Schema" link and a code editor containing the text "true". A yellow box labeled "3" points to the "Example Value" field.

Try to invoke POST /todo

Swagger UI

localhost:8080/ToDoRESTServiceOpenApiSwagger/rest/api-docs?url=/ToDoRESTServiceOpenApiSwagger/rest/openapi.json#/

Parameters

No parameters

Request body required

The description of the input parameter goes here!

Example Value | Schema

```
{
  "id": 0,
  "summary": "string",
  "description": "string"
}
```

JSON request body

Responses

Code	Description	Links
default	The description of the Response goes here!	No links

Media type

application/json

Controls Accept header.

Example Value | Schema

```
true
```

Try to invoke POST /todo

The screenshot shows the Swagger UI interface in a web browser. The address bar displays the URL: `localhost:8080/ToDoRESTServiceOpenApiSwagger/rest/api-docs?url=/ToDoRESTServiceOpenApiSwagger/rest/openapi.json/#/`. The main area shows a JSON schema for a todo item:

```
{
  "id": 1,
  "summary": "string",
  "description": "string"
}
```

An orange arrow points from a circled '1' to the `"id": 1` field. Below the schema is an **Execute** button. Under the **Responses** section, the **Curl** tab shows the following command:

```
curl -X 'POST' \
  'http://localhost:8080/ToDoRESTServiceOpenApiSwagger/rest/todo' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 1,
    "summary": "string",
    "description": "string"
  }'
```

The **Request URL** is `http://localhost:8080/ToDoRESTServiceOpenApiSwagger/rest/todo`. The **Server response** section shows a **Code** of 200. The **Response body** is `false`. An orange arrow points from the `false` response to a code block on the right.

```
@Override
public boolean create(Todo todo) {
    List<Todo> todos = TodoDAO.getTodos();
    if (todos.contains(todo)) {
        return false;
    }
    return todos.add(todo);
}
```

At the bottom right, there is a **Download** button.

Try to invoke POST /todo

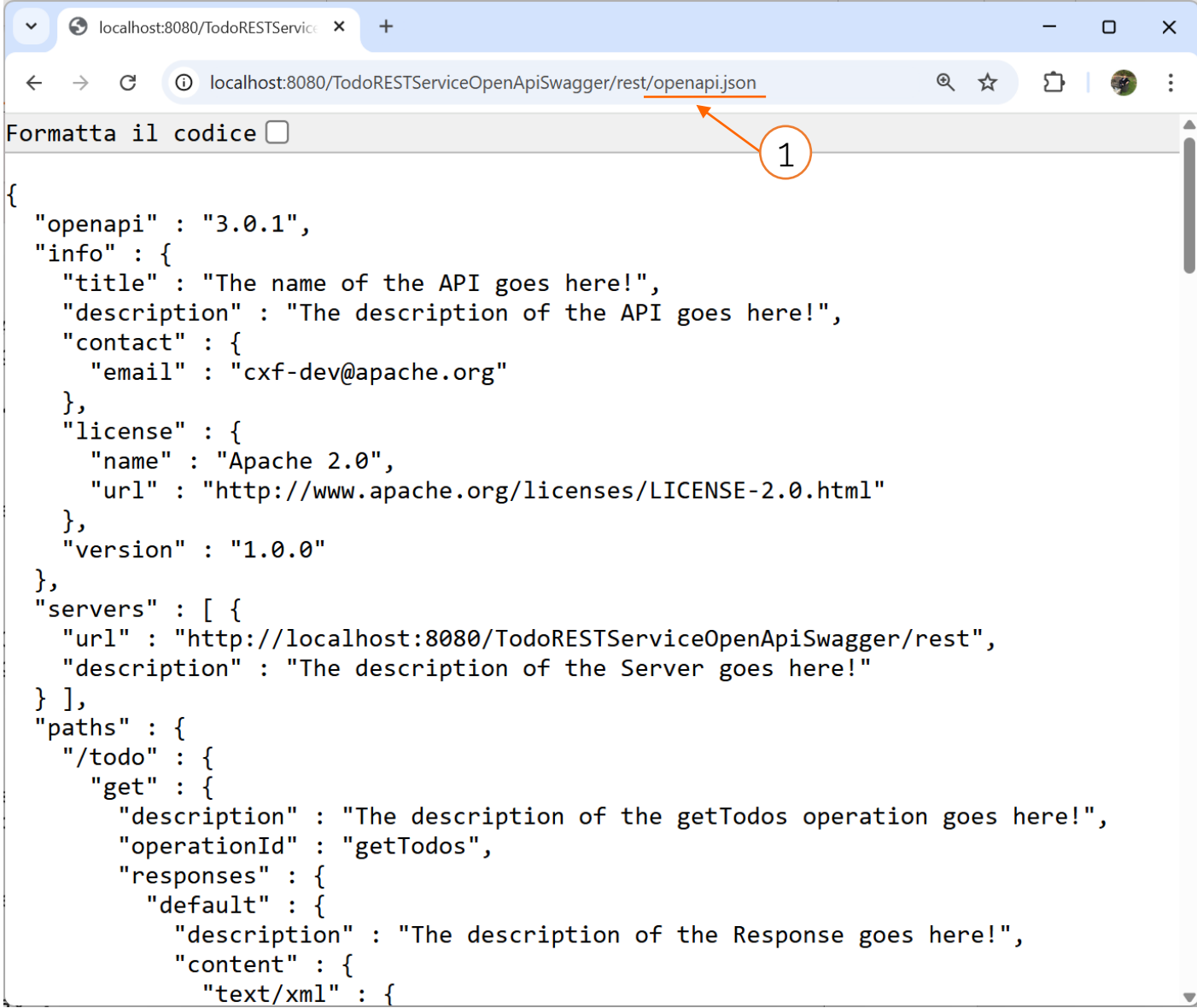
The screenshot shows the Swagger UI interface in a web browser. The address bar displays the URL: `localhost:8080/ToDoRESTServiceOpenApiSwagger/rest/api-docs?url=/ToDoRESTServiceOpenApiSwagger/rest/openapi.json#/`. The main area shows a JSON body for a POST request: `{ "id": 10, "summary": "string", "description": "string" }`. A red circle with the number '1' is around the `"id": 10,` line, with an arrow pointing to it. Below the body is an `Execute` button. The `Responses` section shows a `200` status code with a response body of `true`. A red arrow points from the `true` in the response body to a code snippet in a white box. The code snippet is:

```
@Override
public boolean create(Todo todo) {
    List<Todo> todos = TodoDAO.getTodos();
    if (todos.contains(todo)) {
        return false;
    }
    return todos.add(todo);
}
```

The `return todos.add(todo);` line in the code snippet is highlighted with a dashed orange border, and a red arrow points from it to the `true` in the response body.

Generate a “jaxrs-cxf-client”
for our REST service

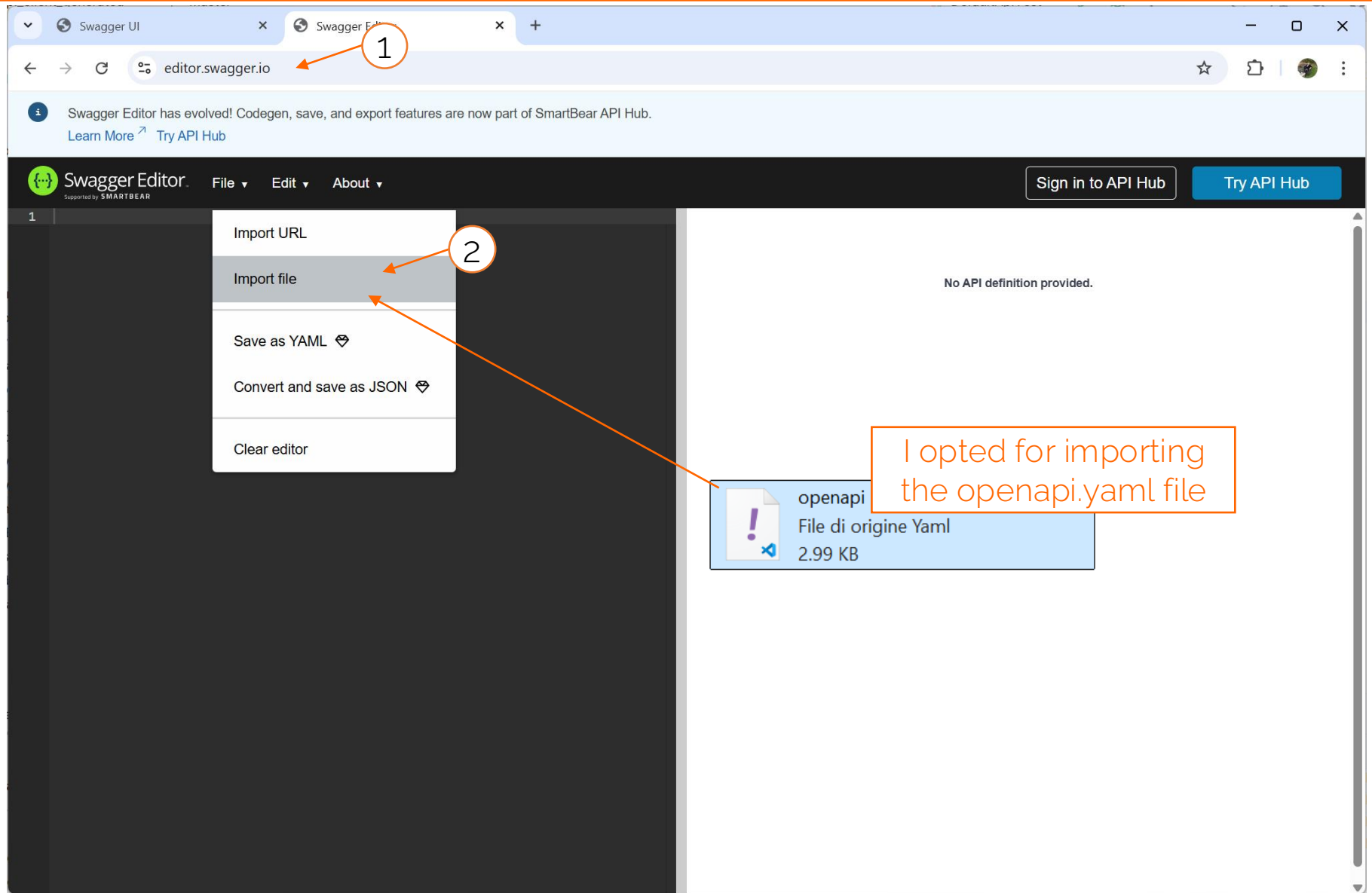
Save the JSON or YAML Description



The screenshot shows a web browser window with the address bar displaying `localhost:8080/ToDoRESTServiceOpenApiSwagger/rest/openapi.json`. The page content is a JSON file named `openapi.json`. A red circle with the number "1" is drawn around the `openapi.json` part of the URL, with an arrow pointing to it from the left. The JSON content is as follows:

```
{
  "openapi" : "3.0.1",
  "info" : {
    "title" : "The name of the API goes here!",
    "description" : "The description of the API goes here!",
    "contact" : {
      "email" : "cxf-dev@apache.org"
    },
    "license" : {
      "name" : "Apache 2.0",
      "url" : "http://www.apache.org/licenses/LICENSE-2.0.html"
    },
    "version" : "1.0.0"
  },
  "servers" : [ {
    "url" : "http://localhost:8080/ToDoRESTServiceOpenApiSwagger/rest",
    "description" : "The description of the Server goes here!"
  } ],
  "paths" : {
    "/todo" : {
      "get" : {
        "description" : "The description of the getTodos operation goes here!",
        "operationId" : "getTodos",
        "responses" : {
          "default" : {
            "description" : "The description of the Response goes here!",
            "content" : {
              "text/xml" : {
```

Import the JSON or YAML Description



Import the JSON or YAML Description

The screenshot displays the Swagger Editor web application. The left pane contains a YAML description of an API. The right pane provides a visual representation of the API, including the title, version, description, and a list of endpoints.

YAML Description (Left Pane):

```
1 openapi: 3.0.1
2 info:
3   title: The name of the API goes here!
4   description: The description of the API goes here!
5   contact:
6     email: cxf-dev@apache.org
7   license:
8     name: Apache 2.0
9     url: http://www.apache.org/licenses/LICENSE-2.0.html
10  version: 1.0.0
11 servers:
12 - url: http://localhost:8080/ToDoRESTServiceOpenApiSwagger/rest
13   description: The description of the Server goes here!
14 paths:
15   /todo:
16     get:
17       description: The description of the getTodos operation goes here!
18       operationId: getTodos
19       responses:
20         default:
21           description: The description of the Response goes here!
22           content:
23             text/xml:
24               schema:
25                 type: array
26                 items:
27                   $ref: "#/components/schemas/ToDo"
28             application/xml:
29               schema:
30                 type: array
31                 items:
32                   $ref: "#/components/schemas/ToDo"
33             application/json:
34               schema:
35                 type: array
36                 items:
37                   $ref: "#/components/schemas/ToDo"
38     put:
39       operationId: update
40       requestBody:
41         content:
42           application/json:
43             schema:
```

Visual API Preview (Right Pane):

- Title:** The name of the API goes here! (1.0.0)
- Version:** OAS 3.0
- Description:** The description of the API goes here!
- Contact:** [Contact the developer](#)
- License:** [Apache 2.0](#)
- Servers:** <http://localhost:8080/ToDoRESTServiceOpenApiSwagger/rest> - The description of the !
- Endpoints:**
 - default**
 - GET** /todo
 - PUT** /todo
 - POST** /todo
 - GET** /todo/{id}

Generate the jaxrs-cxf-client Project

The screenshot shows the Swagger Editor interface in a web browser. The left pane displays the OpenAPI specification in YAML format. The right pane shows the visual representation of the API, including the title 'The name of the API goes here!', version '1.0.0', description, and a list of endpoints under the 'default' server. A red circle with the number '1' highlights the 'Generate Client' menu item in the top navigation bar. A tooltip points to this menu item with the text 'Not available in hosted editor. Try API Hub Trial'. An orange callout box with the text 'This feature is not available for free anymore' is also present.

```
1 openapi: 3.0.1
2 info:
3   title: The name of the API goes here!
4   description: The description of the API goes here!
5   contact:
6     email: cxf-dev@apache.org
7   license:
8     name: Apache 2.0
9     url: http://www.apache.org/licenses/LICENSE-2.0.html
10  version: 1.0.0
11  servers:
12    - url: http://localhost:8080/ToDoRESTServiceOpenApiSwagger/rest
13      description: The description of the Server goes here!
14  paths:
15    /todo:
16      get:
17        description: The description of the getTodos operation goes here!
18        operationId: getTodos
19        responses:
20          default:
21            description: The description of the Response goes here!
22            content:
23              text/xml:
24                schema:
25                  type: array
26                  items:
27                    $ref: '#/components/schemas/ToDo'
28              application/xml:
29                schema:
30                  type: array
31                  items:
32                    $ref: '#/components/schemas/ToDo'
33              application/json:
34                schema:
35                  type: array
36                  items:
37                    $ref: '#/components/schemas/ToDo'
38      put:
39        operationId: update
40        requestBody:
41          content:
```

Swagger Editor has evolved! Codegen, save, and export features are now part of SmartBear API Hub.
[Learn More](#) [Try API Hub](#)

File Edit Insert Generate Server Generate Client About

Not available in hosted editor.
[Try API Hub Trial](#)

1 This feature is not available for free anymore

The name of the API goes here! 1.0.0

OAS 3.0

The description of the API goes here!

[Contact the developer](#)

Apache 2.0

Servers

http://localhost:8080/ToDoRESTServiceOpenApiSwagger/rest - The description of the

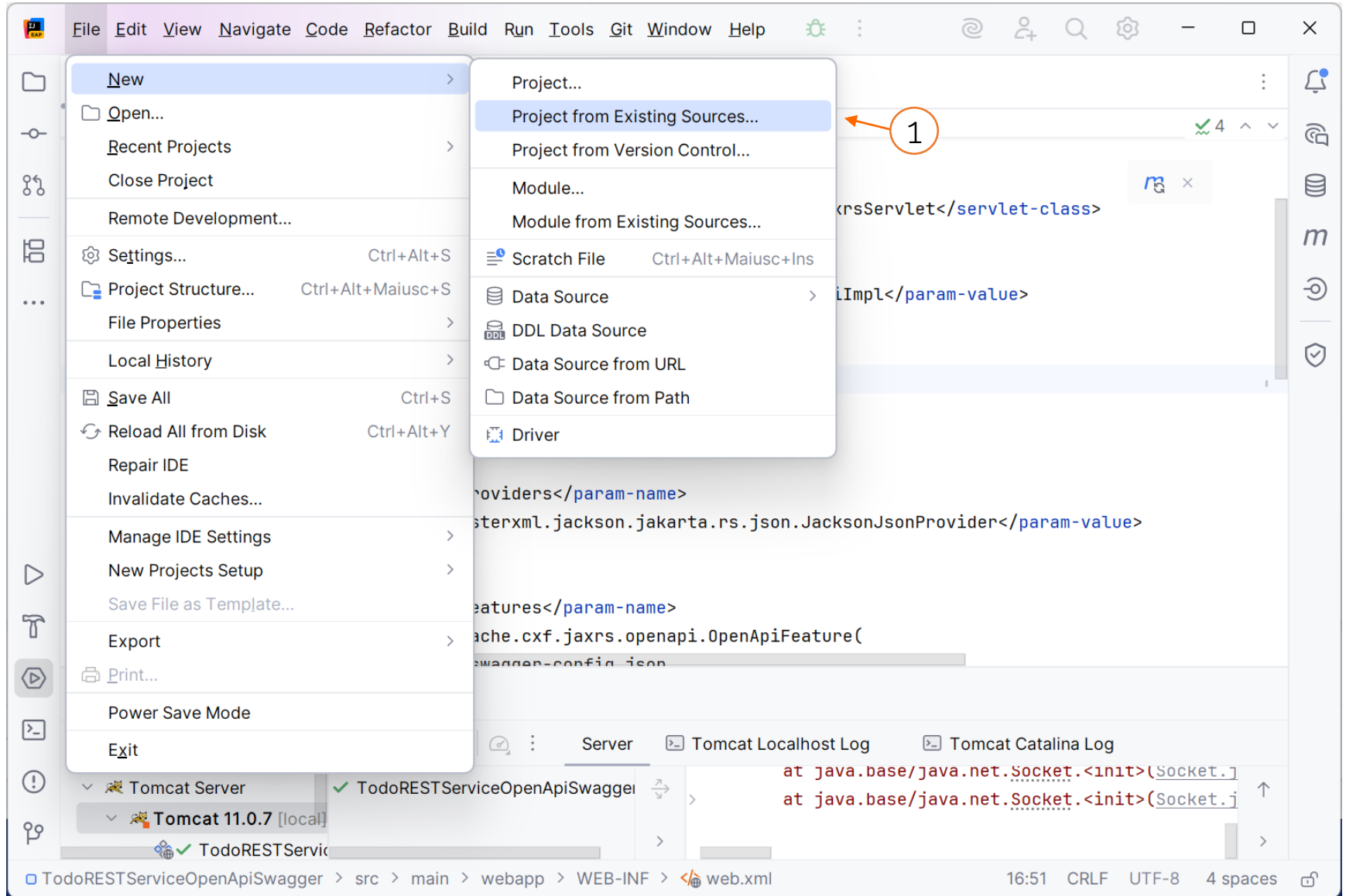
default

GET /todo

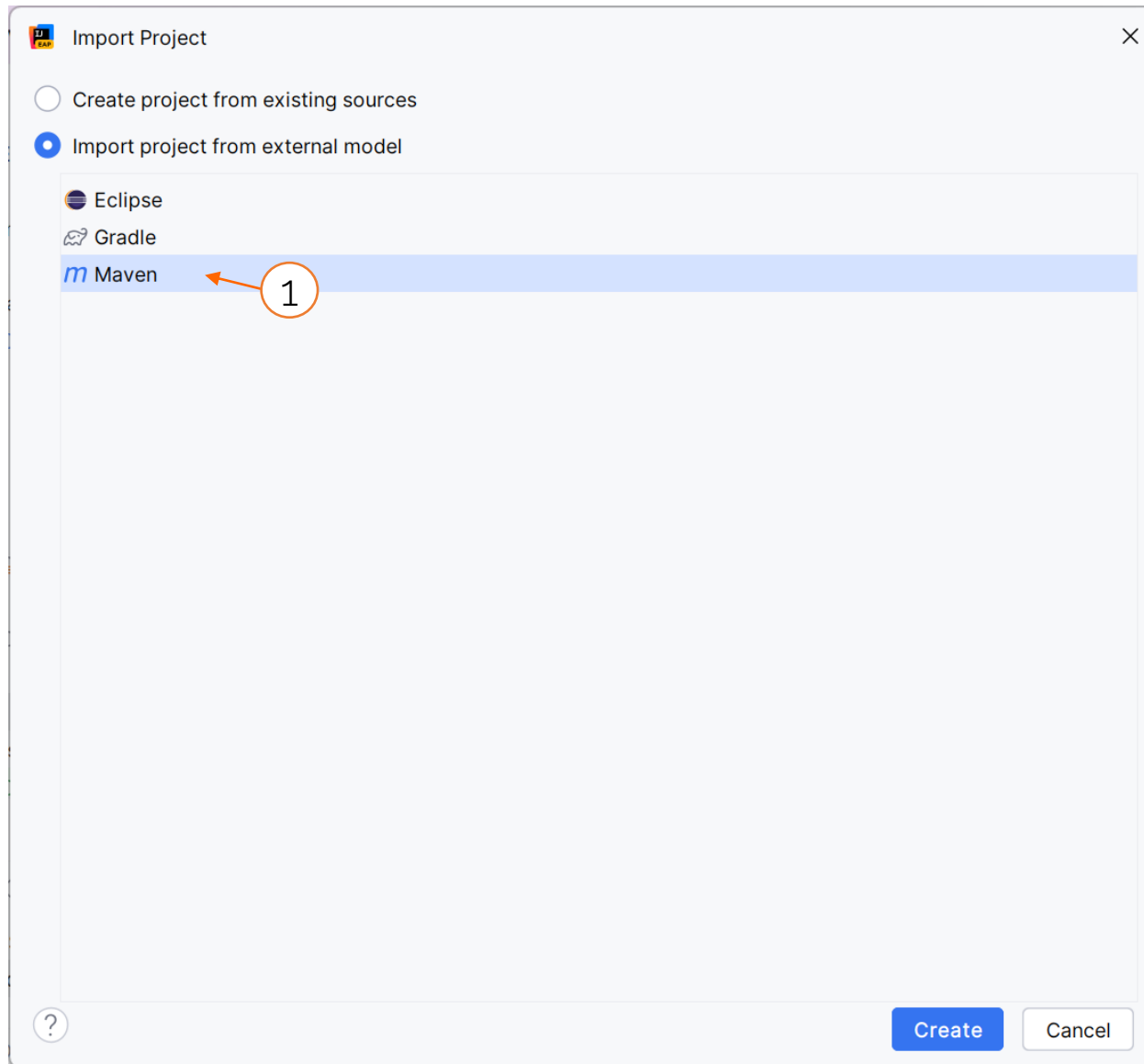
PUT /todo

POST /todo

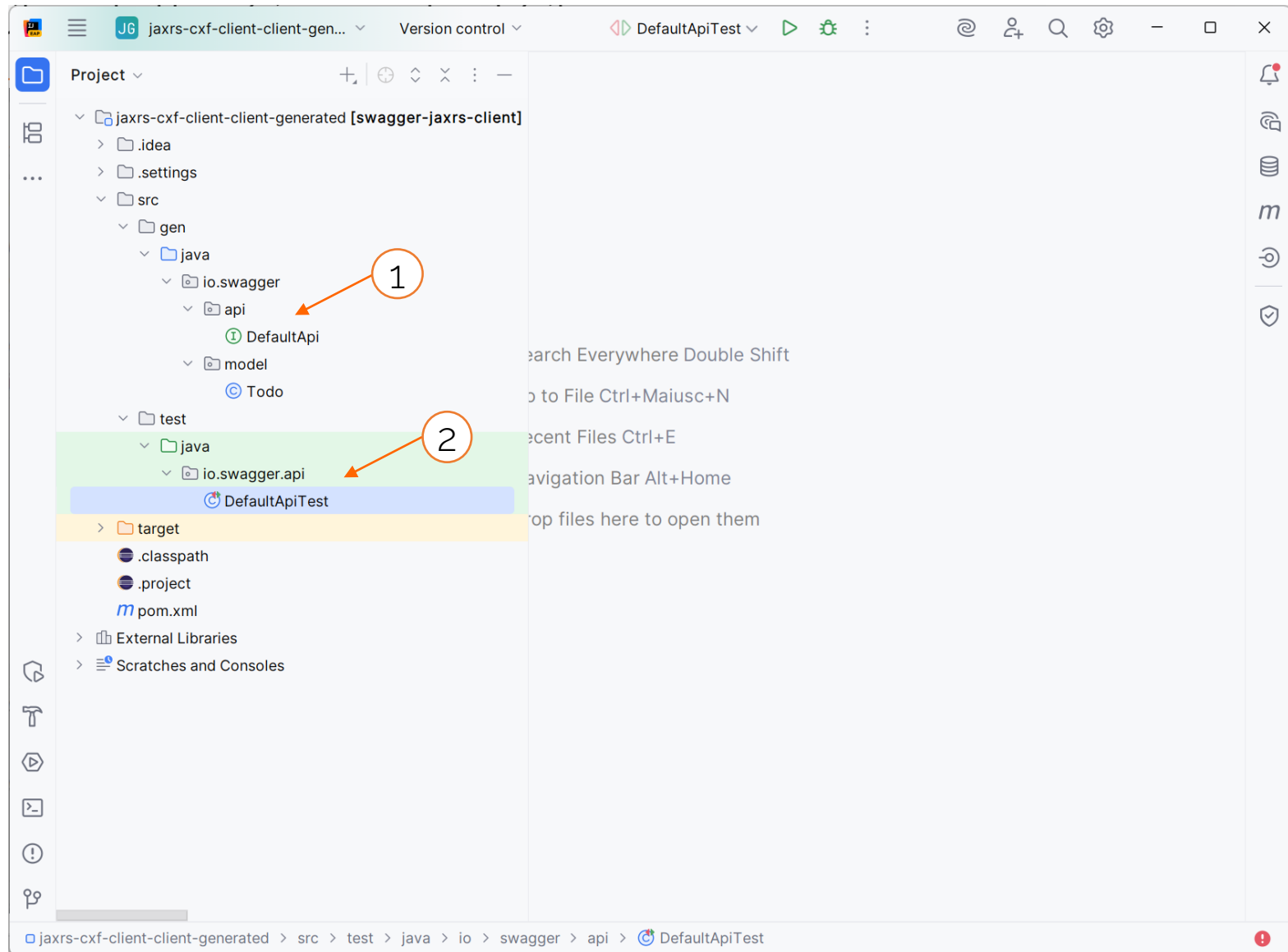
Import the jaxrs-cxf-client Project (1/2)



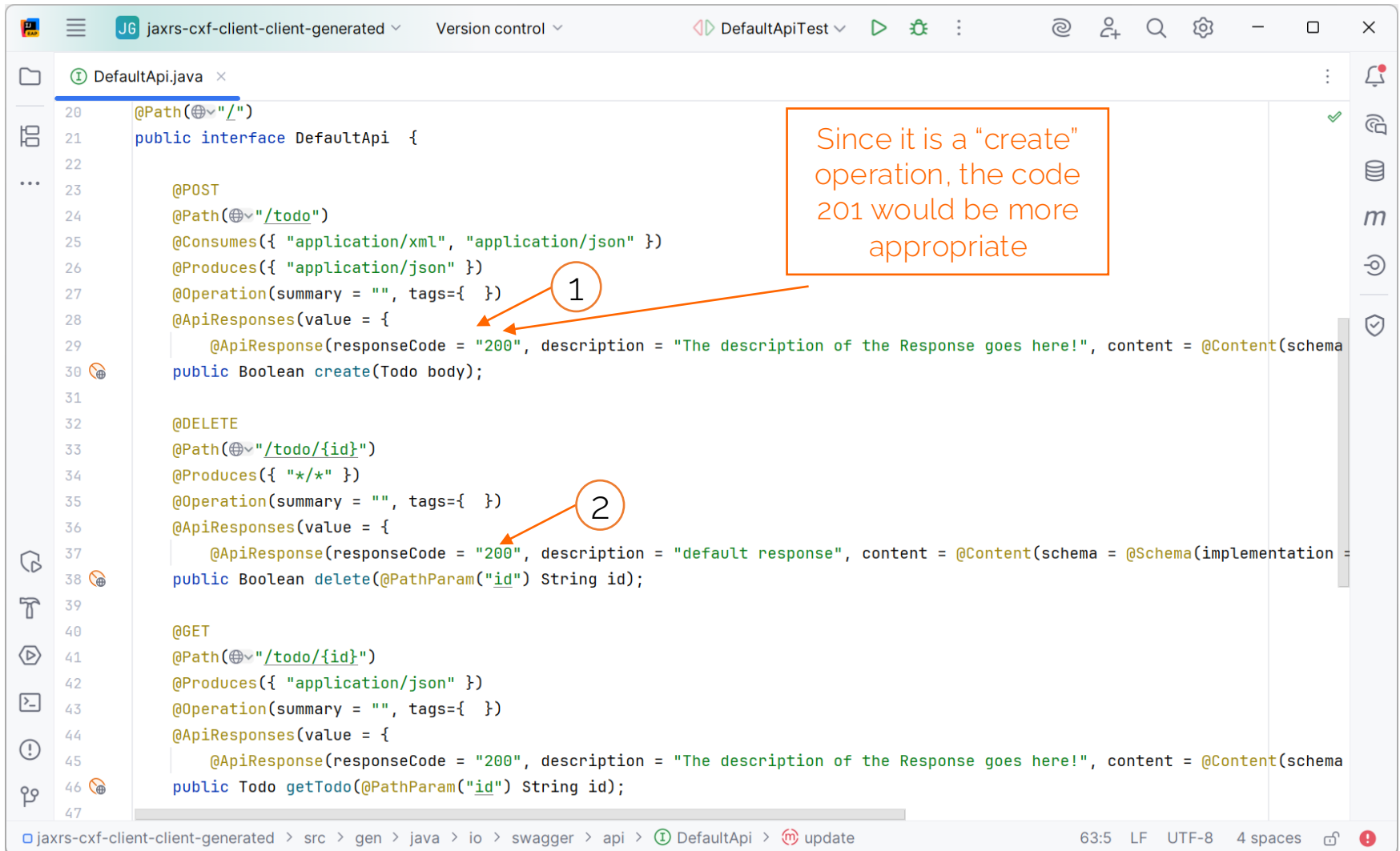
Import the jaxrs-cxf-client Project (2/2)



Check the Generated Classes



Check the Generated Classes

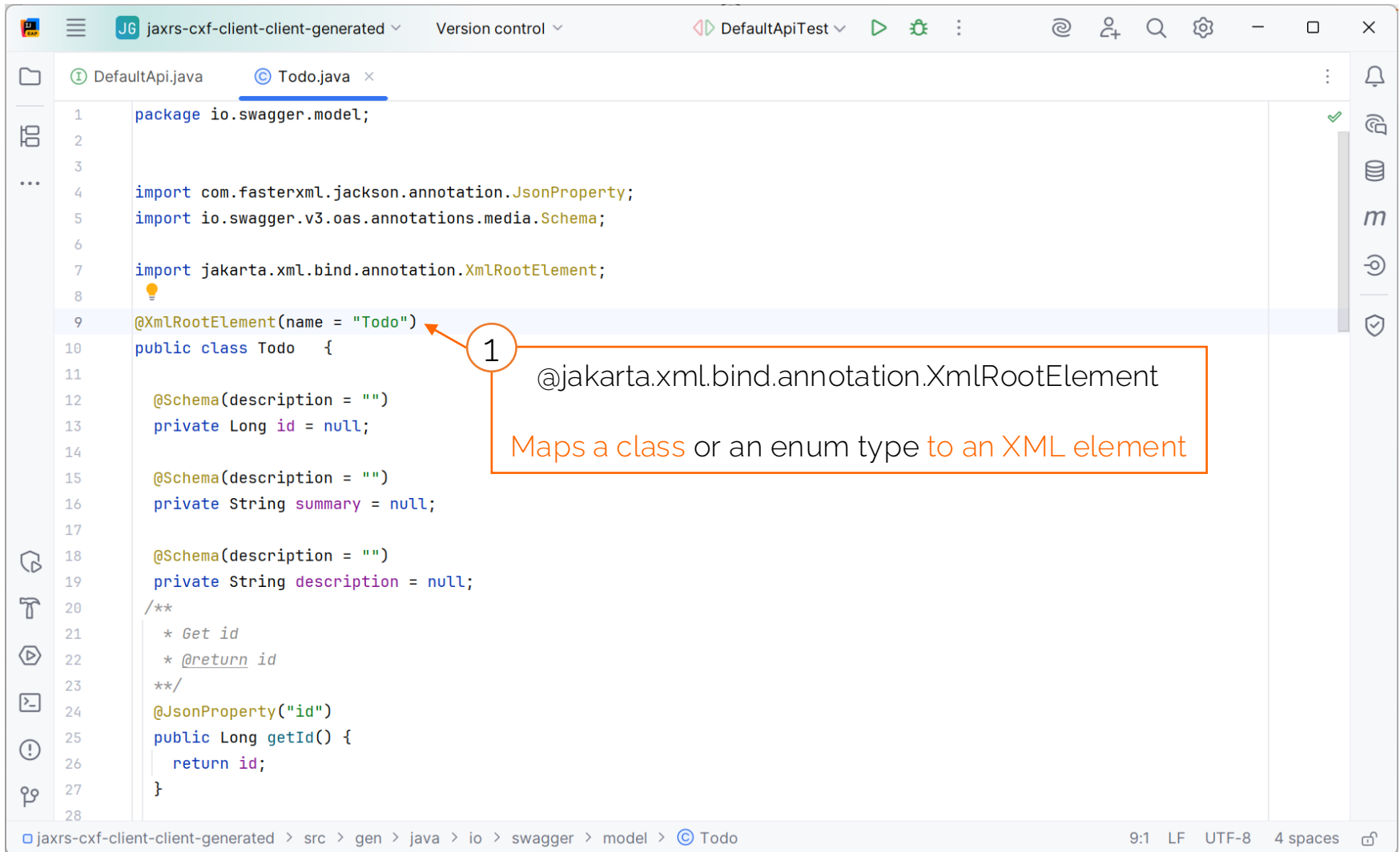


The screenshot shows an IDE window titled "jaxrs-cxf-client-client-generated" with a file named "DefaultApi.java". The code defines a "DefaultApi" interface with three methods: "create", "delete", and "getTodo". The "create" method is annotated with @POST, @Path("/todo"), @Consumes, @Produces, @Operation, and @ApiResponse (responseCode "200"). The "delete" method is annotated with @DELETE, @Path("/todo/{id}"), @Produces, @Operation, and @ApiResponse (responseCode "200"). The "getTodo" method is annotated with @GET, @Path("/todo/{id}"), @Produces, @Operation, and @ApiResponse (responseCode "200"). A callout box with an orange border points to the "200" response code in the "create" method's @ApiResponse annotation, containing the text: "Since it is a 'create' operation, the code 201 would be more appropriate". Two red circles with numbers "1" and "2" are also present, pointing to the "200" response codes in the "create" and "delete" methods respectively.

```
20 @Path("/")
21 public interface DefaultApi {
22
23     @POST
24     @Path("/todo")
25     @Consumes({ "application/xml", "application/json" })
26     @Produces({ "application/json" })
27     @Operation(summary = "", tags={ })
28     @ApiResponse(value = {
29         @ApiResponse(responseCode = "200", description = "The description of the Response goes here!", content = @Content(schema
30 public Boolean create(Todo body);
31
32     @DELETE
33     @Path("/todo/{id}")
34     @Produces({ "*/*" })
35     @Operation(summary = "", tags={ })
36     @ApiResponse(value = {
37         @ApiResponse(responseCode = "200", description = "default response", content = @Content(schema = @Schema(implementation =
38 public Boolean delete(@PathParam("id") String id);
39
40     @GET
41     @Path("/todo/{id}")
42     @Produces({ "application/json" })
43     @Operation(summary = "", tags={ })
44     @ApiResponse(value = {
45         @ApiResponse(responseCode = "200", description = "The description of the Response goes here!", content = @Content(schema
46 public Todo getTodo(@PathParam("id") String id);
47
```

<https://restfulapi.net/http-status-codes/>

Fix the Todo Class



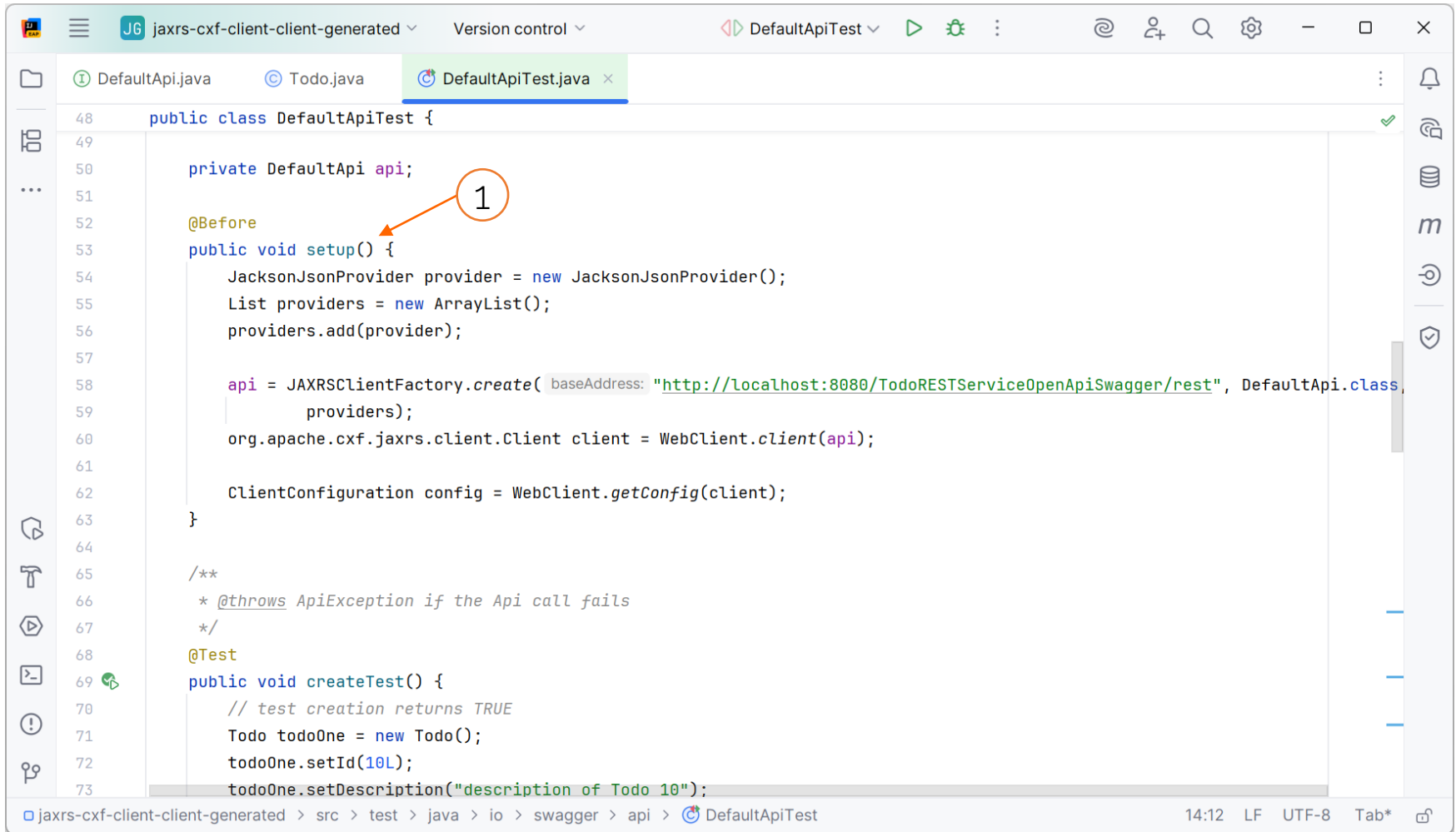
The screenshot shows an IDE window with the file `Todo.java` open. The code defines a `Todo` class with fields `id`, `summary`, and `description`, and a `getId` method. An annotation `@XmlElement(name = "Todo")` is applied to the class. A callout box points to this annotation with the number '1' and contains the following text:

1 `@jakarta.xml.bind.annotation.XmlRootElement`
Maps a class or an enum type to an XML element

```
1 package io.swagger.model;
2
3
4 import com.fasterxml.jackson.annotation.JsonProperty;
5 import io.swagger.v3.oas.annotations.media.Schema;
6
7 import jakarta.xml.bind.annotation.XmlRootElement;
8
9 @XmlElement(name = "Todo")
10 public class Todo {
11
12     @Schema(description = "")
13     private Long id = null;
14
15     @Schema(description = "")
16     private String summary = null;
17
18     @Schema(description = "")
19     private String description = null;
20
21     /**
22      * Get id
23      * @return id
24      */
25     @JsonProperty("id")
26     public Long getId() {
27         return id;
28     }
29 }
```

The IDE interface includes a top bar with project name 'jaxrs-cxf-client-client-generated', version control, and various tool icons. The left sidebar shows a file explorer and a search bar. The bottom status bar indicates the file path 'jaxrs-cxf-client-client-generated > src > gen > java > io > swagger > model > Todo' and encoding 'UTF-8'.

Check the DefaultApiTest Class



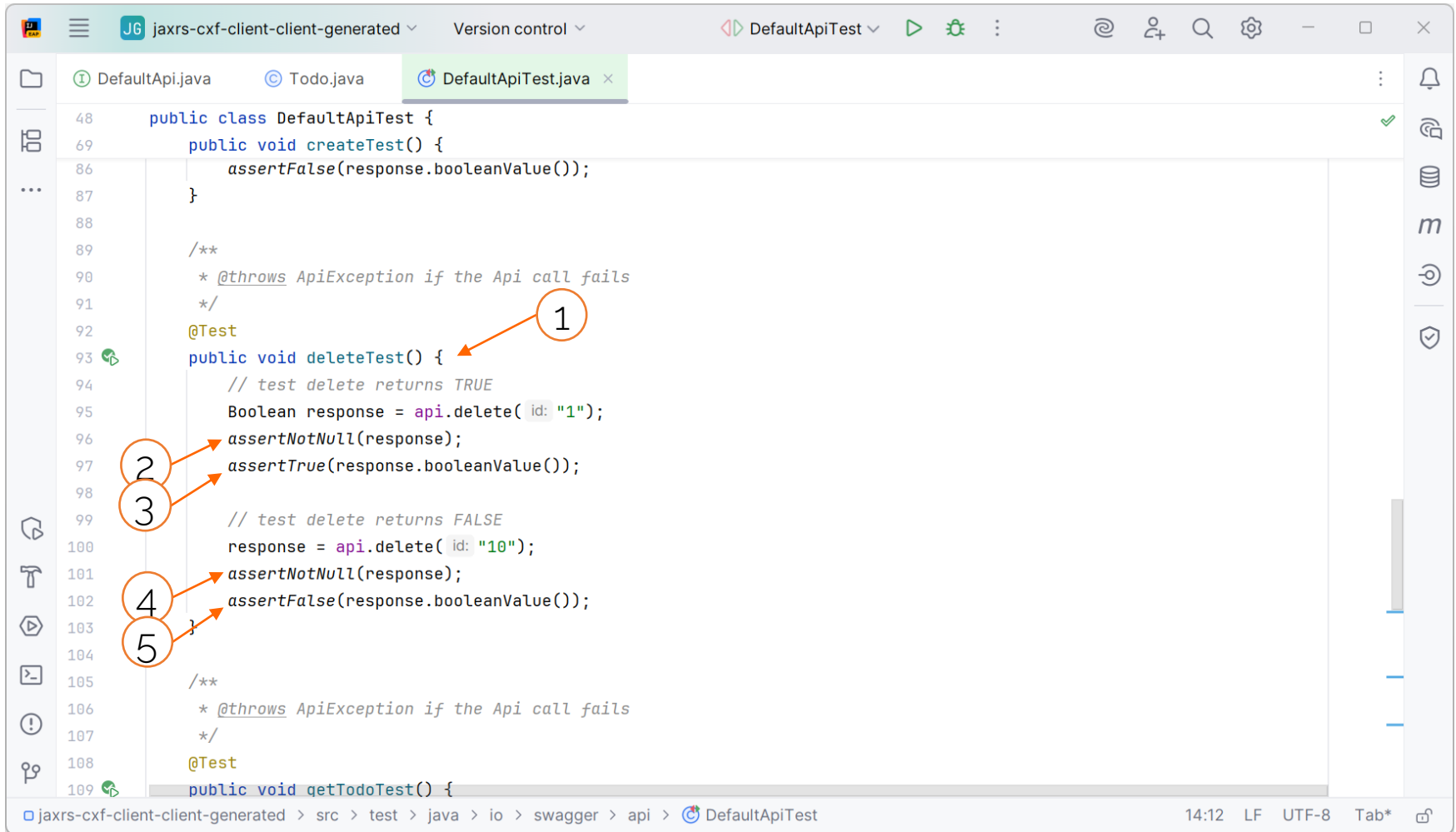
```
48 public class DefaultApiTest {
49
50     private DefaultApi api;
51
52     @Before
53     public void setup() {
54         JacksonJsonProvider provider = new JacksonJsonProvider();
55         List providers = new ArrayList();
56         providers.add(provider);
57
58         api = JAXRSClientFactory.create( baseAddress: "http://localhost:8080/ToDoRESTServiceOpenApiSwagger/rest", DefaultApi.class,
59             providers);
60         org.apache.cxf.jaxrs.client.Client client = WebClient.client(api);
61
62         ClientConfiguration config = WebClient.getConfig(client);
63     }
64
65     /**
66      * @throws ApiException if the Api call fails
67      */
68     @Test
69     public void createTest() {
70         // test creation returns TRUE
71         Todo todoOne = new Todo();
72         todoOne.setId(10L);
73         todoOne.setDescription("description of Todo 10");
74     }
75 }
```

The screenshot shows an IDE window with the file 'DefaultApiTest.java' open. The code defines a 'DefaultApiTest' class with a 'private DefaultApi api;' field. The '@Before' annotated 'setup()' method initializes the 'api' field using 'JAXRSClientFactory.create()'. A red circle with the number '1' is drawn around the '@Before' annotation, with an arrow pointing to the 'setup()' method. The '@Test' annotated 'createTest()' method is also visible. The IDE interface includes a top bar with project and file names, a left sidebar with navigation icons, and a bottom status bar showing file path and encoding.

Implement the createTest

```
48 public class DefaultApiTest {
64
65     /**
66      * @throws ApiException if the Api call fails
67      */
68     @Test
69     public void createTest() {
70         // test creation returns TRUE
71         Todo todoOne = new Todo();
72         todoOne.setId(10L);
73         todoOne.setDescription("description of Todo 10");
74         todoOne.setSummary("summary of Todo 10");
75         Boolean response = api.create(todoOne);
76         assertNotNull(response);
77         assertTrue(response.booleanValue());
78
79         // test creation returns FALSE
80         Todo todoTwo = new Todo();
81         todoTwo.setId(1L);
82         todoTwo.setDescription("description of Todo 1");
83         todoTwo.setSummary("summary of Todo 1");
84         response = api.create(todoTwo);
85         assertNotNull(response);
86         assertFalse(response.booleanValue());
87     }
88 }
```

Implement the deleteTest



```
48 public class DefaultApiTest {
69     public void createTest() {
86         assertFalse(response.booleanValue());
87     }
88
89     /**
90      * @throws ApiException if the Api call fails
91      */
92     @Test
93     public void deleteTest() {
94         // test delete returns TRUE
95         Boolean response = api.delete(id: "1");
96         assertNotNull(response);
97         assertTrue(response.booleanValue());
98
99         // test delete returns FALSE
100        response = api.delete(id: "10");
101        assertNotNull(response);
102        assertFalse(response.booleanValue());
103    }
104
105    /**
106     * @throws ApiException if the Api call fails
107     */
108    @Test
109    public void getTodoTest() {
```

Implement the getTodoTest

```
48 public class DefaultApiTest {
104
105     /**
106      * @throws ApiException if the Api call fails
107      */
108     @Test
109     public void getTodoTest() {
110         // test getTodo returns Todo with ID = 1
111         Todo response = api.getTodo( id: "1");
112         assertNotNull(response);
113         assertEquals( expected: 1L, response.getId().longValue());
114
115         // test getTodo returns null
116         response = api.getTodo( id: "10");
117         assertNull(response);
118     }
119
120
121
122
123
124
125
126
127
128 }
```

1

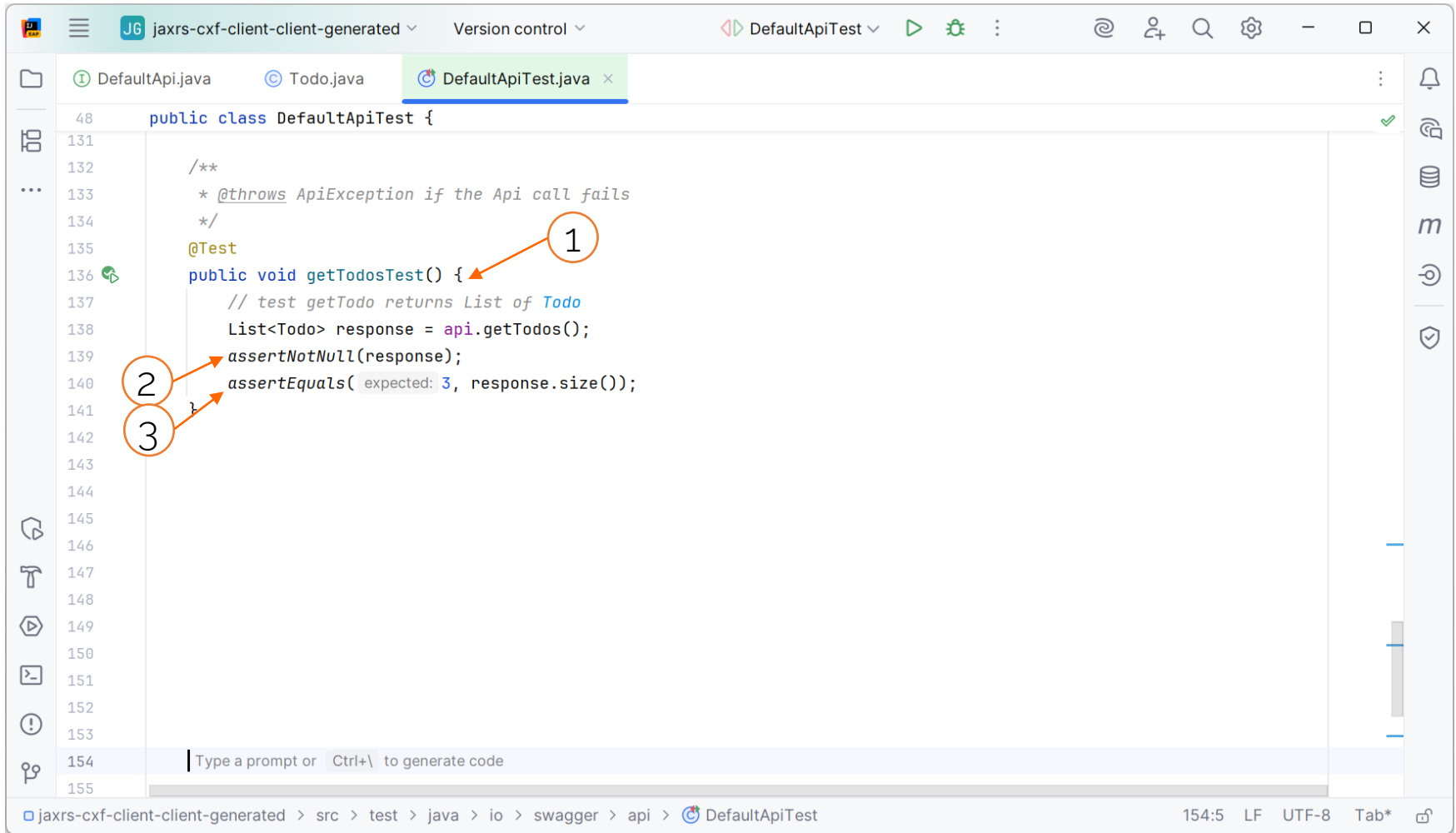
2

3

4

jaxrs-cxf-client-client-generated > src > test > java > io > swagger > api > DefaultApiTest 131:5 LF UTF-8 Tab*

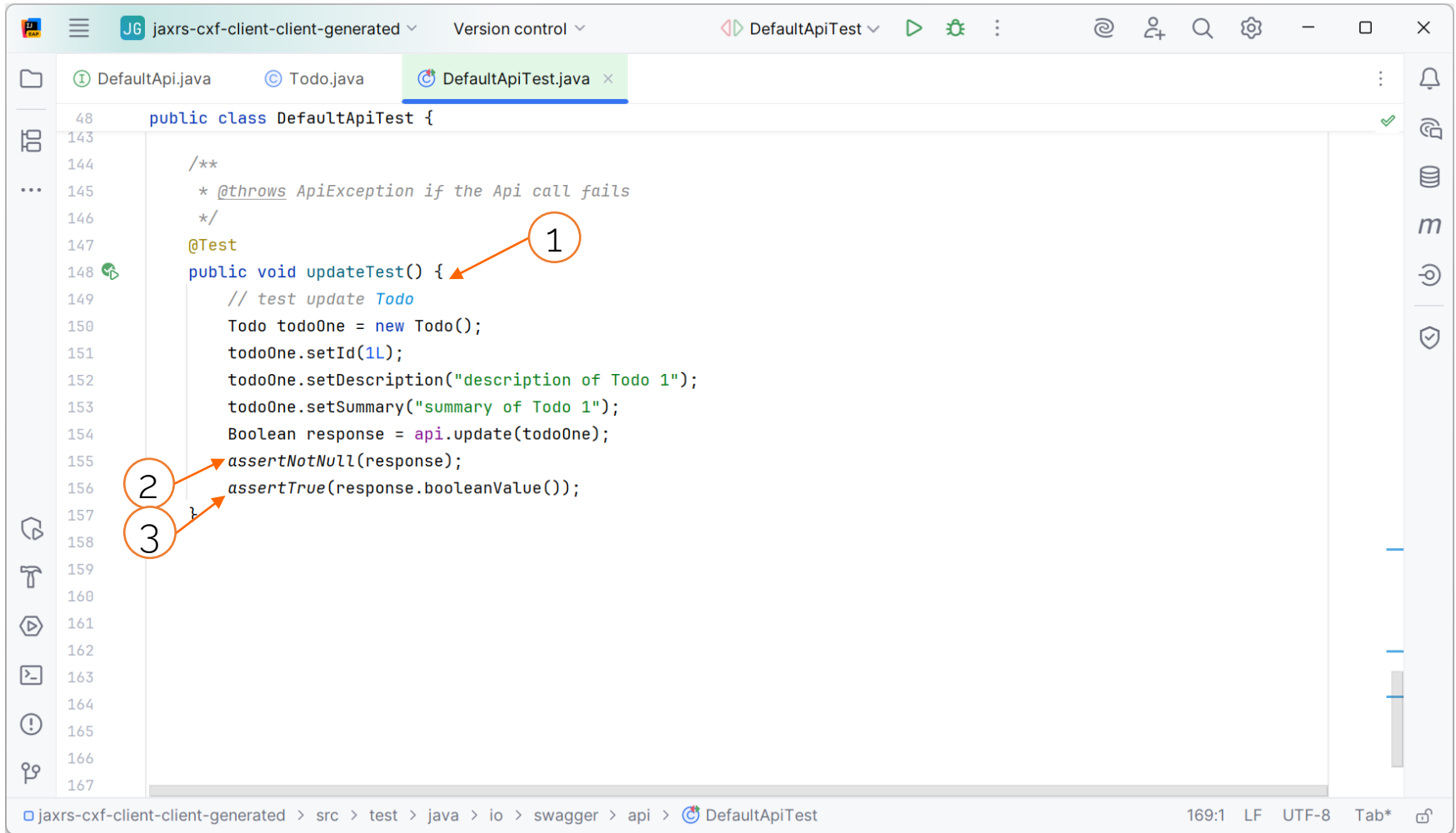
Implement the getTodosTest



```
48 public class DefaultApiTest {
131
132     /**
133      * @throws ApiException if the Api call fails
134      */
135     @Test
136     public void getTodosTest() {
137         // test getTodo returns List of Todo
138         List<Todo> response = api.getTodos();
139         assertNotNull(response);
140         assertEquals( expected: 3, response.size());
141     }
142
143
144
145
146
147
148
149
150
151
152
153
154 | Type a prompt or Ctrl+I to generate code
155
```

The screenshot shows an IDE window for the file `DefaultApiTest.java`. The code defines a `public class DefaultApiTest` with a `@Test` annotated method `getTodosTest()`. The method calls `api.getTodos()` and performs two assertions: `assertNotNull(response)` and `assertEquals(3, response.size())`. Three orange circles with numbers 1, 2, and 3 are overlaid on the code. Circle 1 points to the `@Test` annotation. Circle 2 points to the `api.getTodos()` call. Circle 3 points to the `assertEquals` call. The IDE interface includes a top bar with project and version control information, a left sidebar with file explorer and tool icons, and a bottom status bar showing the current file path and encoding.

Implement the updateTest

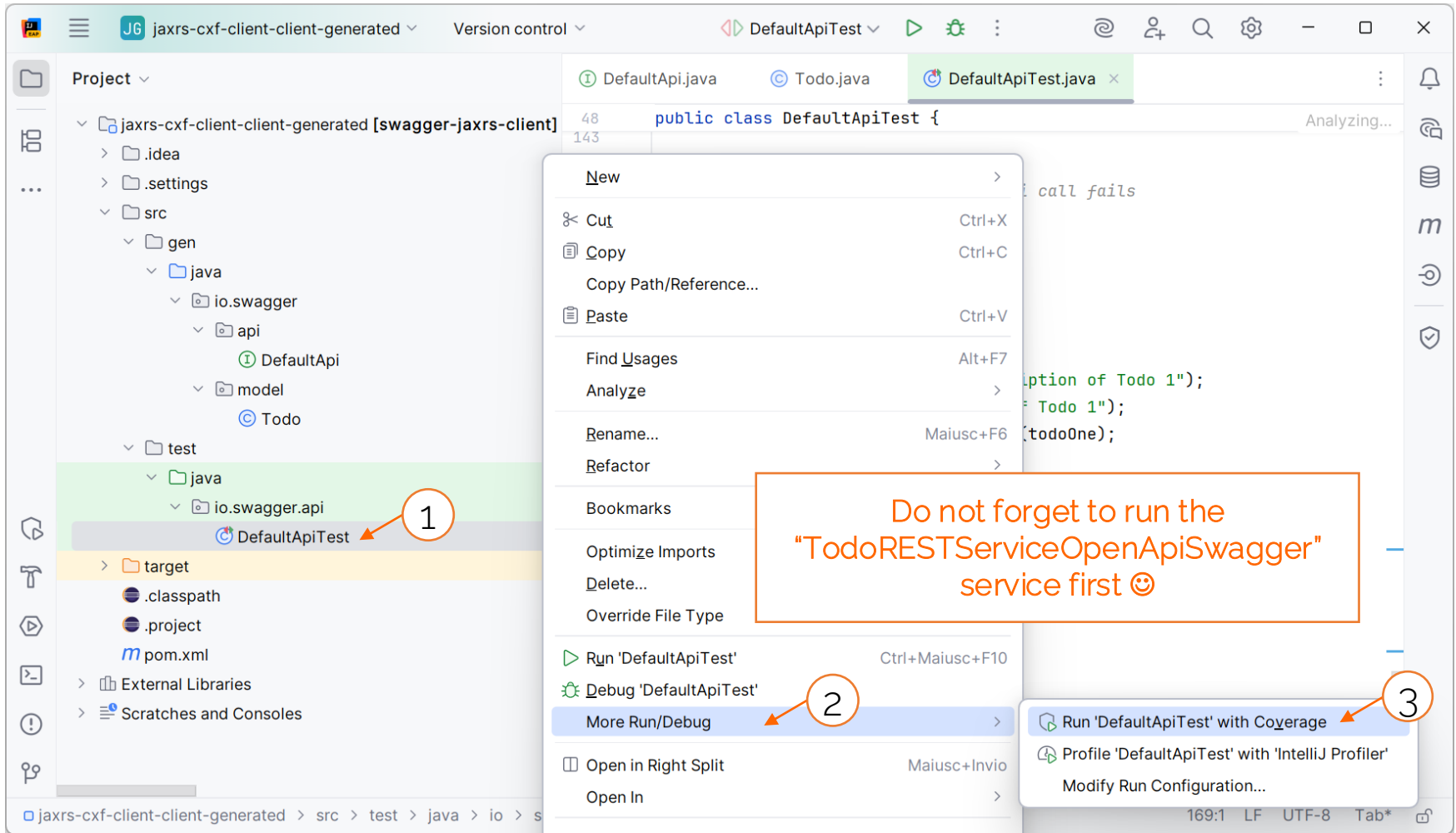


```
48 public class DefaultApiTest {
143
144     /**
145      * @throws ApiException if the Api call fails
146      */
147     @Test
148     public void updateTest() {
149         // test update Todo
150         Todo todoOne = new Todo();
151         todoOne.setId(1L);
152         todoOne.setDescription("description of Todo 1");
153         todoOne.setSummary("summary of Todo 1");
154         Boolean response = api.update(todoOne);
155         assertNotNull(response);
156         assertTrue(response.booleanValue());
157     }
158
159
160
161
162
163
164
165
166
167
```

Annotations in the image:

- 1: Points to the `updateTest()` method signature.
- 2: Points to the `assertNotNull(response);` assertion.
- 3: Points to the `assertTrue(response.booleanValue());` assertion.

Execute Tests



Check Tests Execution

The screenshot shows an IDE with the following components:

- Editor:** Displays `DefaultApiTest.java` with the following code:

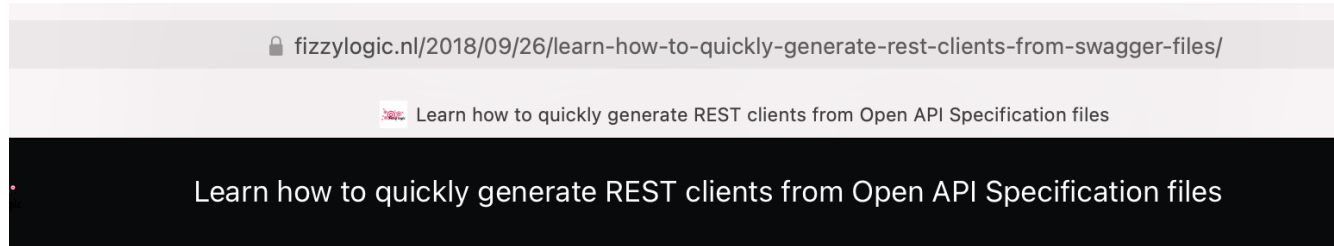
```
1 package io.swagger.model;  
2  
3  
4 import com.fasterxml.jackson.annotation.JsonProperty;  
5 import io.swagger.v3.oas.annotations.media.Schema;  
6  
7 import jakarta.xml.bind.annotation.XmlRootElement;  
8  
9 @XmlRootElement(name = "Todo")  
10 public class Todo {  
11
```
- Coverage Panel:** Shows coverage for `DefaultApiTest` with 100% coverage across all elements.
- Test Results Panel:** Shows the execution of `DefaultApiTest` with 5 tests passed. The tests and their durations are:
 - ✓ `getTodosTest` (685 ms)
 - ✓ `updateTest` (122 ms)
 - ✓ `getTodoTest` (41 ms)
 - ✓ `createTest` (31 ms)
 - ✓ `deleteTest` (15 ms)
- Log Panel:** Shows the output of the tests, including debug messages from `org.apache.cxf`.

Annotations in the image:

- 1:** Points to the `DefaultApiTest` tab in the Test Results panel.
- 2:** Points to the `5 tests passed` status in the Test Results panel.

Generating the REST Client

That's the method we have used so far for generating the REST Client...



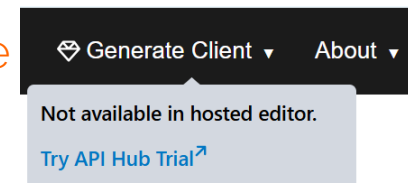
Method 1: Use the swagger editor

The most interactive way to create a client from a swagger file is using the online swagger editor. Go to <https://editor.swagger.io/>. Select **file**, **import URL** and type in the URL of the swagger endpoint you want to load.

Alternatively you can select **File**, **Import File** and upload the downloaded swagger.json file.

Next select **Generate client** and choose the language of your choice. The end result is a zip file you can download with the generated client code.

... but now unfortunately it is not available for free anymore



<https://fizzylogic.nl/2018/09/26/learn-how-to-quickly-generate-rest-clients-from-swagger-files/>

Alternative Way for Generating the REST Client (1/3)

To use the **openapi-generator** follow these steps:

1. Clone repository <https://github.com/OpenAPITools/openapi-generator/>
2. Navigate the just downloaded git repository with **cd openapi-generator**

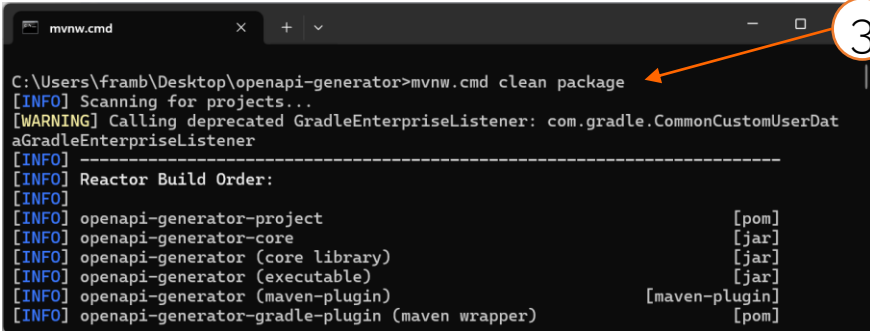
```
C:\Users\framb\Desktop>git clone https://github.com/OpenAPITools/openapi-generator --depth 1
Cloning into 'openapi-generator'...
remote: Enumerating objects: 35891, done.
remote: Counting objects: 100% (35891/35891), done.
remote: Compressing objects: 100% (16713/16713), done.
remote: Total 35891 (delta 20979), reused 25219 (delta 16465), pack-reused 0 (from 0)
Receiving objects: 100% (35891/35891), 24.35 MiB | 4.20 MiB/s, done.
Resolving deltas: 100% (20979/20979), done.
Updating files: 100% (57288/57288), done.

C:\Users\framb\Desktop>cd openapi-generator
```

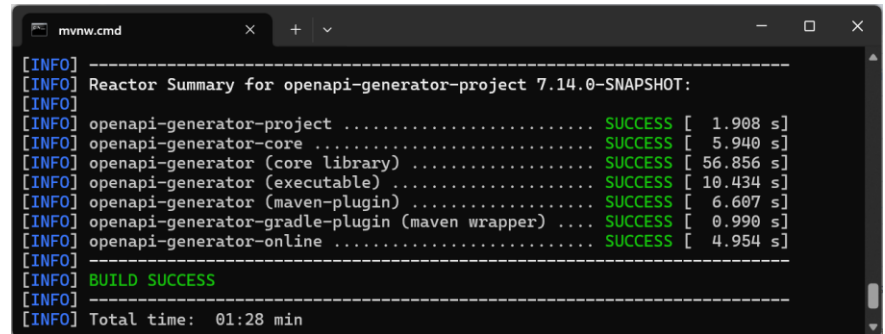


3. Run **mvn clean package** (or mvnw to use maven wrapper inside the repository)

```
mvnw.cmd
C:\Users\framb\Desktop\openapi-generator>mvnw.cmd clean package
[INFO] Scanning for projects...
[WARNING] Calling deprecated GradleEnterpriseListener: com.gradle.CommonCustomUserDataGradleEnterpriseListener
[INFO] -----
[INFO] Reactor Build Order:
[INFO]
[INFO] openapi-generator-project [pom]
[INFO] openapi-generator-core [jar]
[INFO] openapi-generator (core library) [jar]
[INFO] openapi-generator (executable) [jar]
[INFO] openapi-generator (maven-plugin) [maven-plugin]
[INFO] openapi-generator-gradle-plugin (maven wrapper) [pom]
```



```
mvnw.cmd
[INFO] -----
[INFO] Reactor Summary for openapi-generator-project 7.14.0-SNAPSHOT:
[INFO]
[INFO] openapi-generator-project ..... SUCCESS [ 1.908 s]
[INFO] openapi-generator-core ..... SUCCESS [ 5.940 s]
[INFO] openapi-generator (core library) ..... SUCCESS [ 56.856 s]
[INFO] openapi-generator (executable) ..... SUCCESS [ 10.434 s]
[INFO] openapi-generator (maven-plugin) ..... SUCCESS [ 6.607 s]
[INFO] openapi-generator-gradle-plugin (maven wrapper) ... SUCCESS [ 0.990 s]
[INFO] openapi-generator-online ..... SUCCESS [ 4.954 s]
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 01:28 min
```



Alternative Way for Generating the REST Client (2/3)

4. Execute this command to generate the client

```
java -jar modules/openapi-generator-cli/target/openapi-generator-cli.jar generate \
-i http://localhost:8080/ToDoRESTServiceOpenApiSwagger/rest/openapi.json \
-g java \
-o /java_api_client
```

1

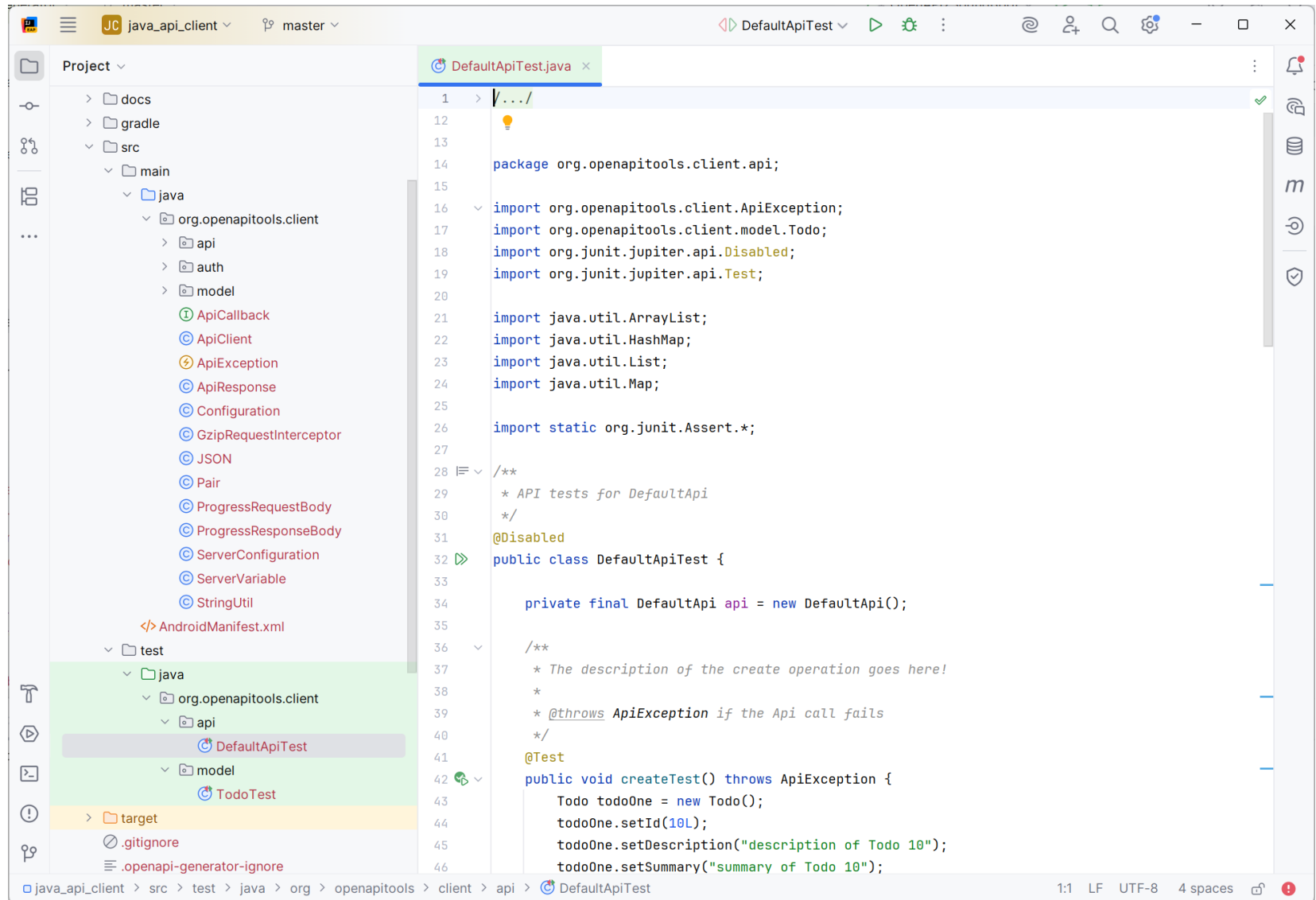
2

3

There are three arguments for this command:

- -i Specifies the location (URL or file path) of the OpenAPI specification
- -g Defines the target language or platform for code generation.
- -o Sets the directory where the generated code will be saved.

Alternative Way for Generating the REST Client (3/3)



The screenshot shows an IDE window with the project structure on the left and the code for `DefaultApiTest.java` in the center. The project structure is as follows:

- Project
 - docs
 - gradle
 - src
 - main
 - java
 - org.openapitools.client
 - api
 - auth
 - model
 - ApiCallback
 - ApiClient
 - ApiException
 - ApiResponse
 - Configuration
 - GzipRequestInterceptor
 - JSON
 - Pair
 - ProgressRequestBody
 - ProgressResponseBody
 - ServerConfiguration
 - ServerVariable
 - StringUtil
 - AndroidManifest.xml
 - test
 - java
 - org.openapitools.client
 - api
 - DefaultApiTest
 - model
 - TodoTest
 - target
 - .gitignore
 - .openapi-generator-ignore

The code for `DefaultApiTest.java` is as follows:

```
1  > /.../
12
13
14  package org.openapitools.client.api;
15
16  import org.openapitools.client.ApiException;
17  import org.openapitools.client.model.TODO;
18  import org.junit.jupiter.api.Disabled;
19  import org.junit.jupiter.api.Test;
20
21  import java.util.ArrayList;
22  import java.util.HashMap;
23  import java.util.List;
24  import java.util.Map;
25
26  import static org.junit.Assert.*;
27
28  /**
29   * API tests for DefaultApi
30   */
31  @Disabled
32  public class DefaultApiTest {
33
34      private final DefaultApi api = new DefaultApi();
35
36      /**
37       * The description of the create operation goes here!
38       *
39       * @throws ApiException if the Api call fails
40       */
41      @Test
42      public void createTest() throws ApiException {
43          TODO todoOne = new TODO();
44          todoOne.setId(10L);
45          todoOne.setDescription("description of TODO 10");
46          todoOne.setSummary("summary of TODO 10");
```

Execute Tests with new client

The screenshot displays an IDE interface with the following components:

- Project Explorer:** Shows a project structure with a 'test' directory containing a 'java' subdirectory. The 'DefaultApiTest' class is highlighted under 'org.openapitools.client'.
- Code Editor:** Displays the source code of 'DefaultApiTest.java'. The code includes a package declaration, imports, and two test methods: 'createTest()' and 'deleteTest()'. The 'createTest()' method uses 'assertNotNull' and 'assertTrue' to verify the response.
- Coverage View:** Shows a table of coverage data for the 'DefaultApiTest' class.
- Test Results View:** Displays the results of the test execution, showing that 5 tests passed out of 5 tests total, with a total execution time of 111 ms.

Element	Class,...	Metho...	Line, %	Branc...
org.openapitools.client	54% (6...	62% (27...	76% (15...	50% (23...
DefaultApi	54% (6...	62% (27...	76% (15...	50% (23...

Test Method	Duration
getTodosTest()	79 ms
updateTest()	7 ms
getTodoTest()	7 ms
createTest()	12 ms
deleteTest()	6 ms

Process finished with exit code 0

References

- <http://cxf.apache.org/docs/openapifeature.html>
- <https://swagger.io/docs/specification/about/>
- <https://swagger.io/>
- <https://github.com/OpenAPITools/openapi-generator>
- <https://fizzylogic.nl/2018/09/26/learn-how-to-quickly-generate-rest-clients-from-swagger-files/>
- <https://voicebase.readthedocs.io/en/v3/how-to-guides/swagger-codegen.html#download-swagger-codegen-cli-tool>